



Artificial Intelligence (AI-2002)

Chapter 5: Constraint Satisfaction Problem

Constraint satisfaction problems

- Constraint satisfaction problems are mathematical questions defined as a **set of objects** whose **state must satisfy** a **number of constraints** or limitations. CSPs represent the entities in a problem as a homogeneous collection of finite constraints over variables, which is solved by constraint satisfaction methods.

Constraint Satisfaction Problem

Standard search problem:

state is a “black box”—any old data structure
that supports goal test, eval, successor

CSP:

state is defined by variables X_i with values from domain D_i

goal test is a set of constraints specifying
allowable combinations of values for subsets of variables

Simple example of a **formal representation language**

Allows useful **general-purpose** algorithms with more power
than standard search algorithms

Constraint satisfaction problems

- What is a CSP?
 - **Finite set of variables $V_1, V_2, \dots, V_n$**
 - **Non-empty domain of possible values for each variable $D_{V_1}, D_{V_2}, \dots, D_{V_n}$**
 - **Finite set of constraints $C_1, C_2, \dots, C_m$**
 - **Each constraint C_i limits the values that variables can take, e.g., $V_1 \neq V_2$**
- A *state* is defined as an *assignment* of values to some or all variables.
- *Consistent assignment*: assignment does not violate the constraints.

Constraint satisfaction problems

- An assignment is *complete* when every value is mentioned.
- A *solution* to a CSP is a complete assignment that satisfies all constraints.
- Some CSPs require a solution that maximizes an *objective function*.
- Applications: Scheduling the time of observations on the Hubble Space Telescope, Floor planning, Map coloring, Cryptography

Comparison with formal searches

- So far, Search space states evaluated by heuristics and goal.
- A constraint satisfaction problem (CSP) is a problem that requires its solution within some limitations/conditions also known as constraints.
- For which the typical problems can be converted to CSP and solved.

CSP as a standard search problem

- A CSP can easily be expressed as a standard search problem.
- Incremental formulation
 - **Initial State:** the empty assignment {}.
 - **Successor function:** Assign value to unassigned variable provided that there is not conflict.
 - **Goal test:** the current assignment is complete.
 - **Path cost:** as constant cost for every step.

CSP as a standard search problem

- This is the same for all CSP's !!!
- Solution is found at depth n (if there are n variables).
 - **Hence depth first search can be used.**
- Path is irrelevant, so complete state representation can also be used.
- Branching factor b at the top level is nd .
- $b=(n-l)d$ at depth l , hence $n!d^n$ leaves (only d^n complete assignments).

Converting problems to CSPs

- A problem to be converted to CSP requires the following steps:
- **Step 1:** Create a variable set.(Representing state)
- **Step 2:** Create a domain set.(values)
- **Step 3:** Create a constraint set with variables and domains after considering the constraints.(limitations)

CSP Example1-Sudoku

Rule: Fill all empty Squares so that the numbers 1 to 9 appear once in each row, column and 3x3 box

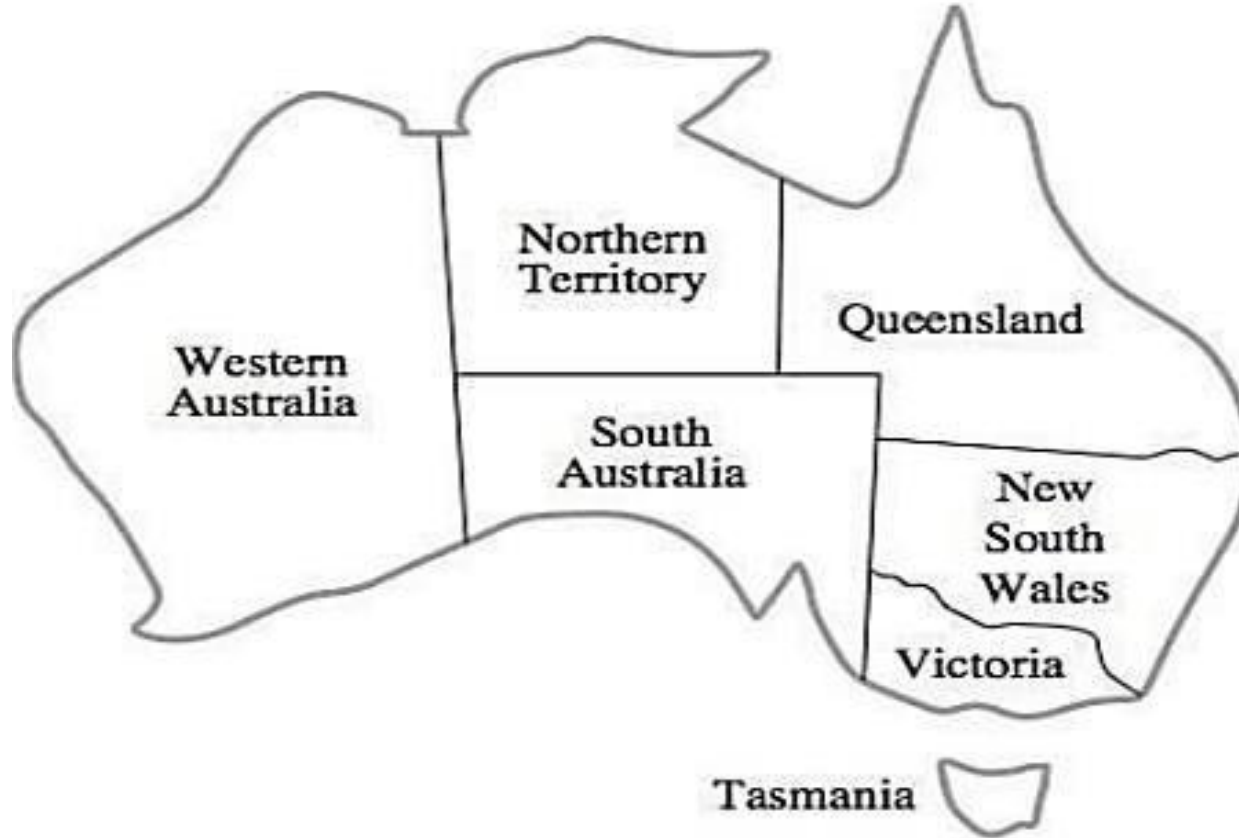
	1	2	3	4	5	6	7	8	9
A			3		2		6		
B	9			3		5			1
C			1	8		6	4		
D			8	1		2	9		
E	7								8
F			6	7		8	2		
G			2	6		9	5		
H	8			2		3			9
I			5		1		3		

VARIABLES – variables are cells (A1-I9)

DOMAIN – domain of each variable is {1,2,3,4,5,6,7,8,9}

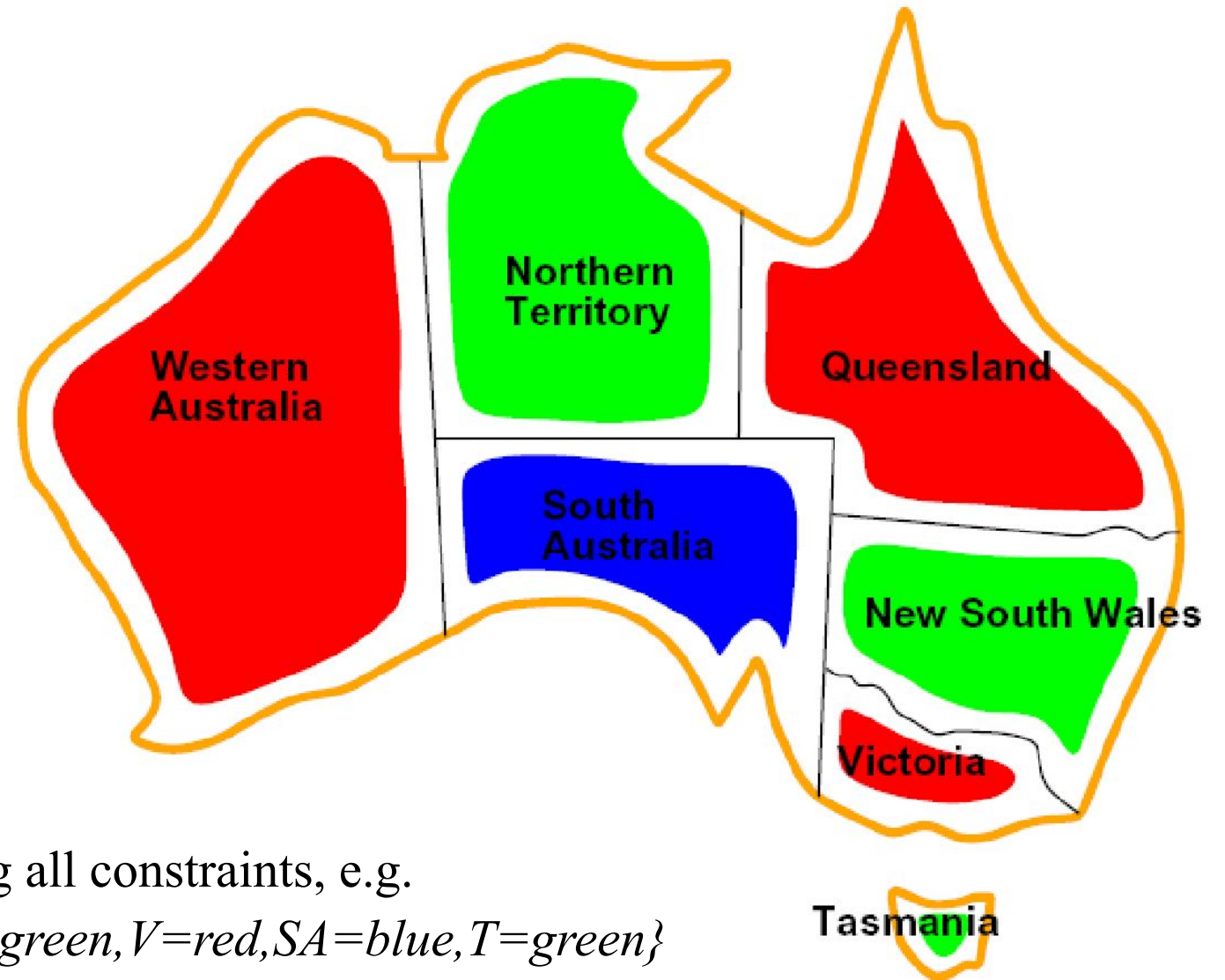
CONSTRAINT– constraints: rows, columns, boxes contain all different numbers

Example2-Map Coloring



- Rule** - Color each region either red, green or blue
- No adjacent region can have the same color

Map Coloring



Solutions are assignments satisfying all constraints, e.g.

$\{WA=red, NT=green, Q=red, NSW=green, V=red, SA=blue, T=green\}$

Map Coloring

Variables:

WA, NT, SA, Q, NSW, V, T

Domains:

$D = \{\text{R}, \text{G}, \text{B}\}$

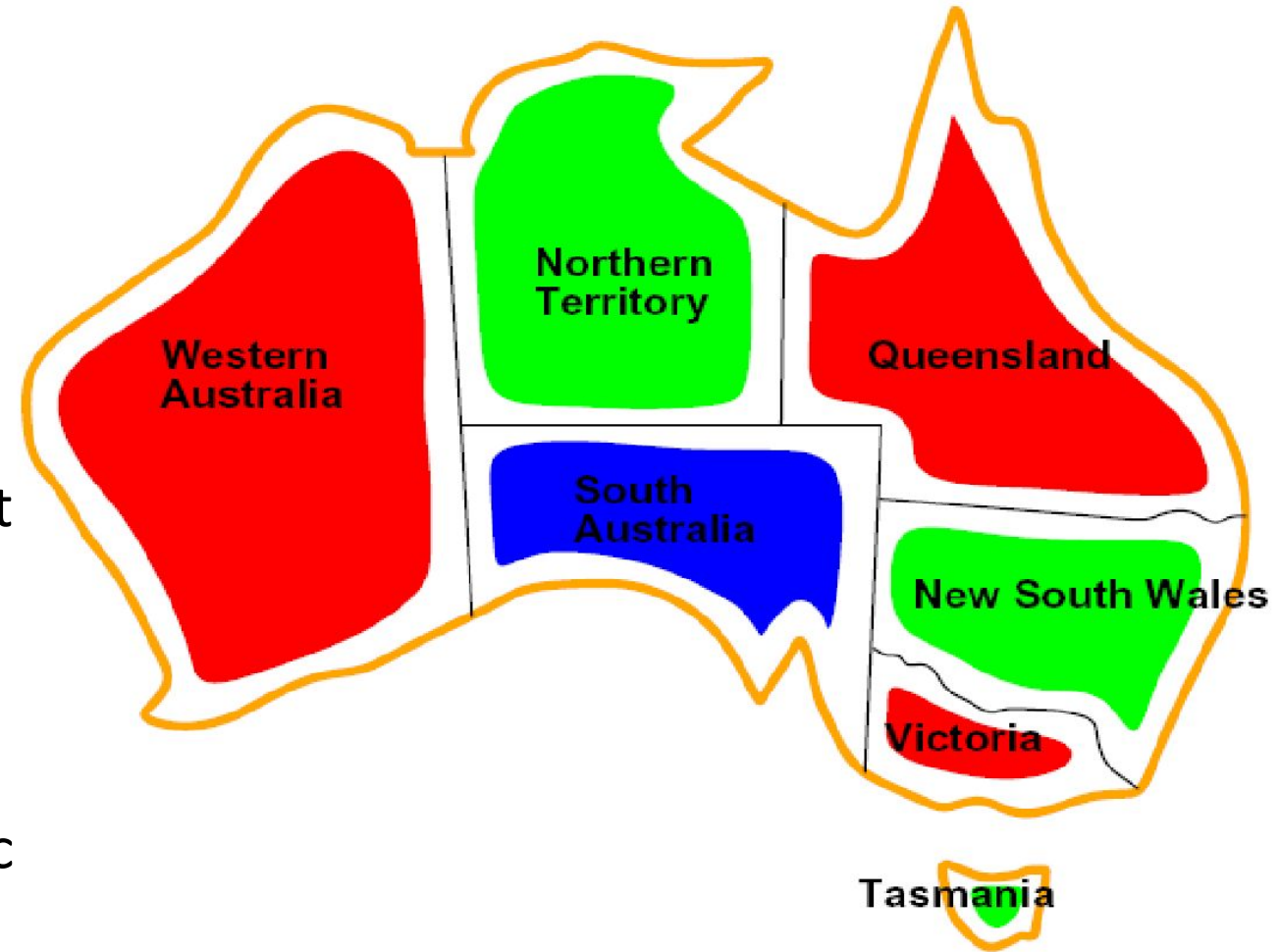
Constraints:

adjacent regions must have different

$WA \neq NT$

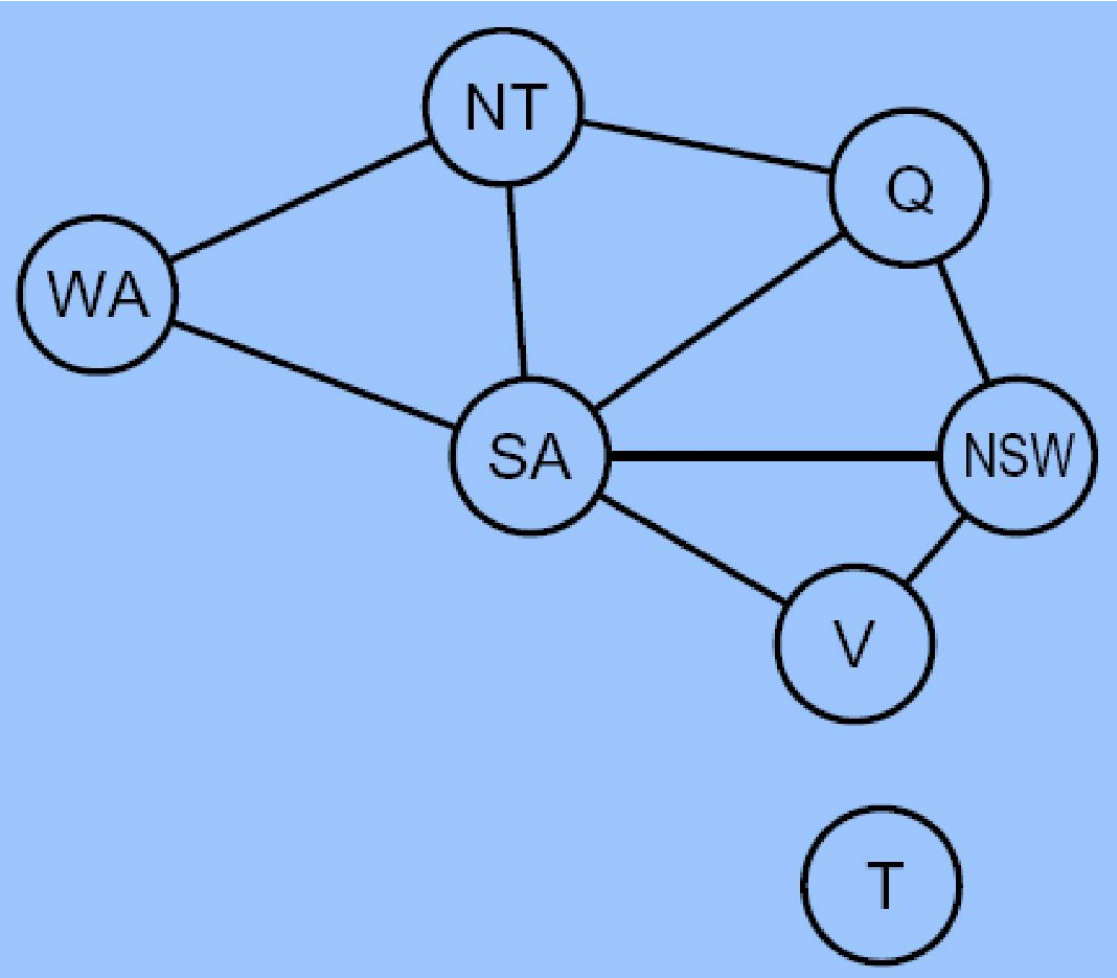
$(WA, NT) \in \{(\text{R}, \text{G}), (\text{R}, \text{B}), (\text{B}, \text{G}), \dots\}$

Solutions are assignments satisfying all c



Constraint Graphs

	WA	NT	SA	Q	NSW	V
WA		1	1	0	0	0
NT	1		1	1	0	0
SA	1	1		1	1	1
Q	0	1	1		1	0
NSW	0	0	1	1	0	1
V	0	0	1	0	1	0

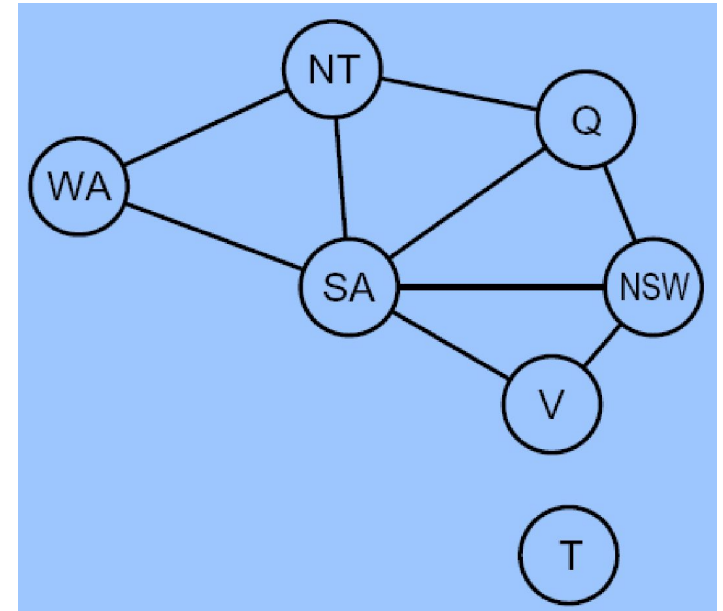


Constraint Graphs

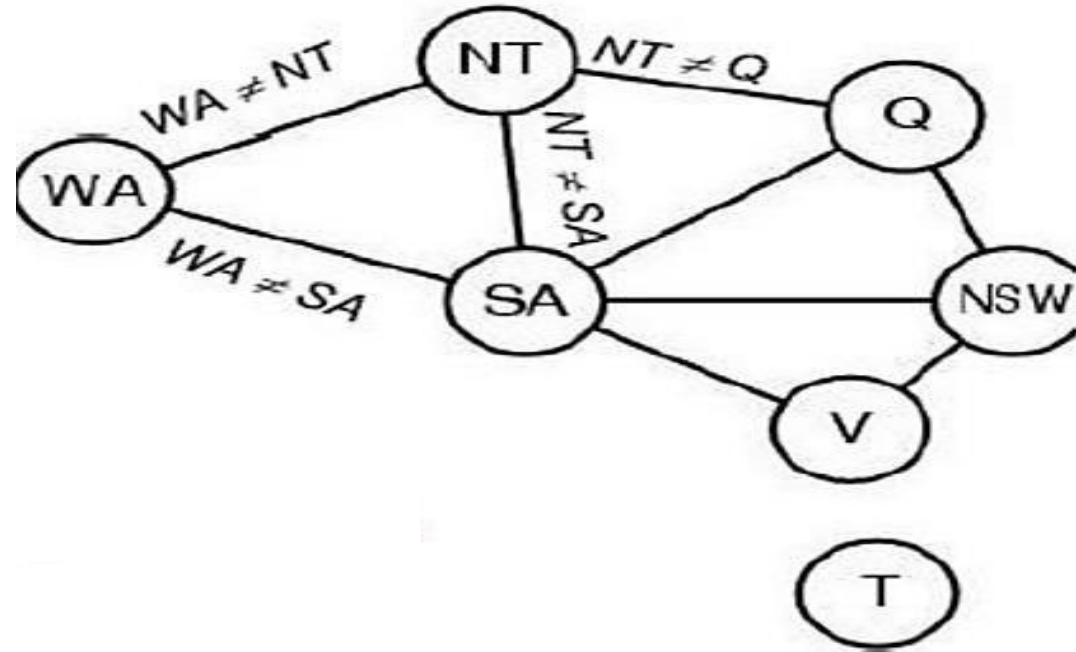
Binary CSP: each constraint relates (at most) two variables

Binary constraint graph: nodes are variables, arcs show constraints

General-purpose CSP algorithms use the graph structure to speed up search. E.g., Tasmania is an independent sub problem!



CSP Representation of Map Coloring



Variables: WA, NT, Q, NSW, V, SA, T

Domains: $D_i = \{\text{red, green, blue}\}$

Constraints: adjacent regions must have different colors

e.g., $WA \neq NT$, or (WA, NT)

Constraint Graph

- **Nodes** are Variables
- **Arcs** are Constraint

Commutativity

- CSPs are commutative.
 - **The order of any given set of actions has no effect on the outcome.**
 - **Example: choose colors for Australian territories one at a time**
 - [WA=red then NT=green] same as [NT=green then WA=red]
 - All CSP search algorithms consider a single variable assignment at a time □ there are d^n leaves.

Varieties of constraints

- **Unary constraints** involve a single variable, e.g. $SA \neq \text{green}$
- **Binary constraints** involve pairs of variables (relates two variables), e.g., $SA \neq WA$
- **Higher-order constraints** involve 3 or more variables, e.g. cryptarithmic puzzles (column constraints)



Backtracking search

- Depth-first search
- Chooses values for one variable at a time and backtracks when a variable has no legal values left to assign.
- Uninformed algorithm
 - **No good general performance**

Backtracking search

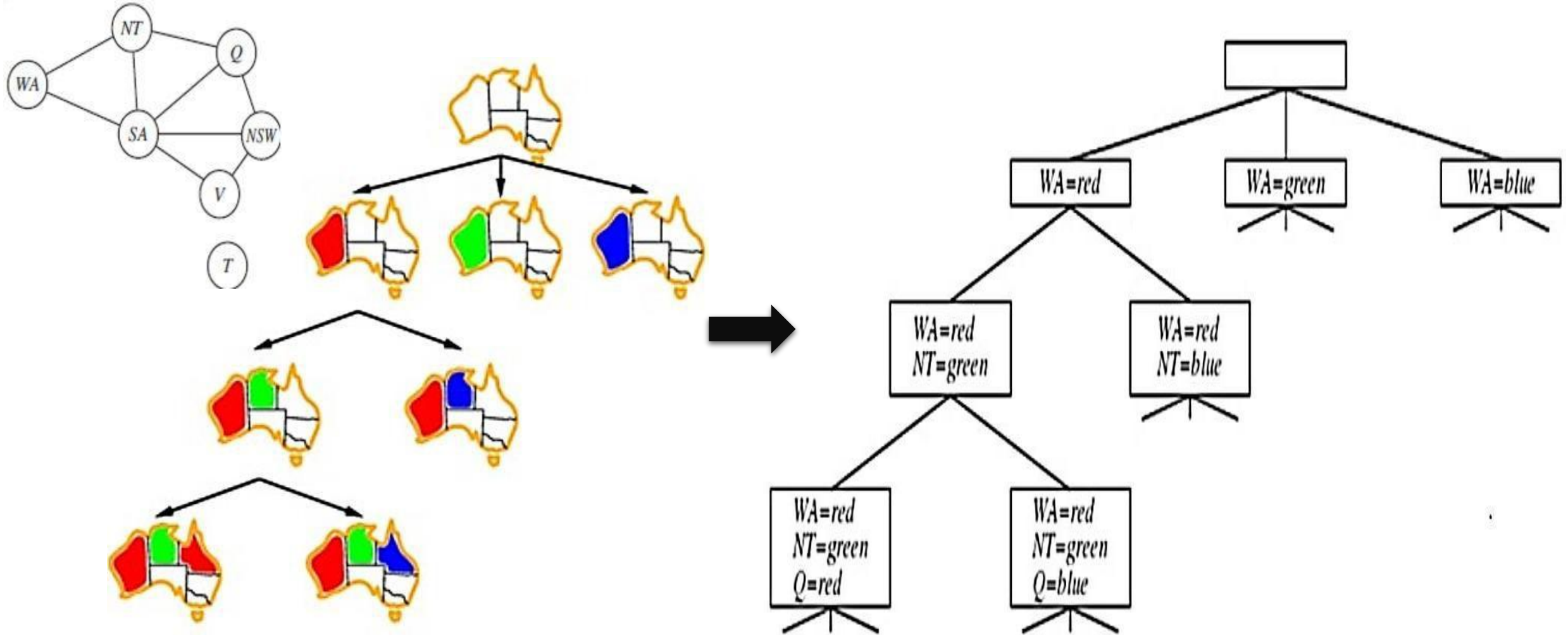
```
function BACKTRACKING-SEARCH(csp) return a solution or failure  
  return RECURSIVE-BACKTRACKING({} , csp)
```

```
function RECURSIVE-BACKTRACKING(assignment, csp) return a solution or failure  
  if assignment is complete then return assignment  
  var  $\leftarrow$  SELECT-UNASSIGNED-VARIABLE(VARIABLES[csp],assignment,csp)  
  for each value in ORDER-DOMAIN-VALUES(var, assignment, csp) do  
    if value is consistent with assignment according to CONSTRAINTS[csp] then  
      add {var=value} to assignment  
      result  $\leftarrow$  RECURSIVE-BACKTRACKING(assignment, csp)  
      if result  $\neq$  failure then return result  
      remove {var=value} from assignment  
  return failure
```

Backtracking search

- Depth-first search for CSPs with single-variable assignments is called backtracking search
- Variable assignments are commutative: [WA = red then NT = green] same as [NT = green then WA = red]
- Only need to consider assignments to a single variable at each node
- Backtracks when variable has no legal value

Backtracking for the map-coloring problem

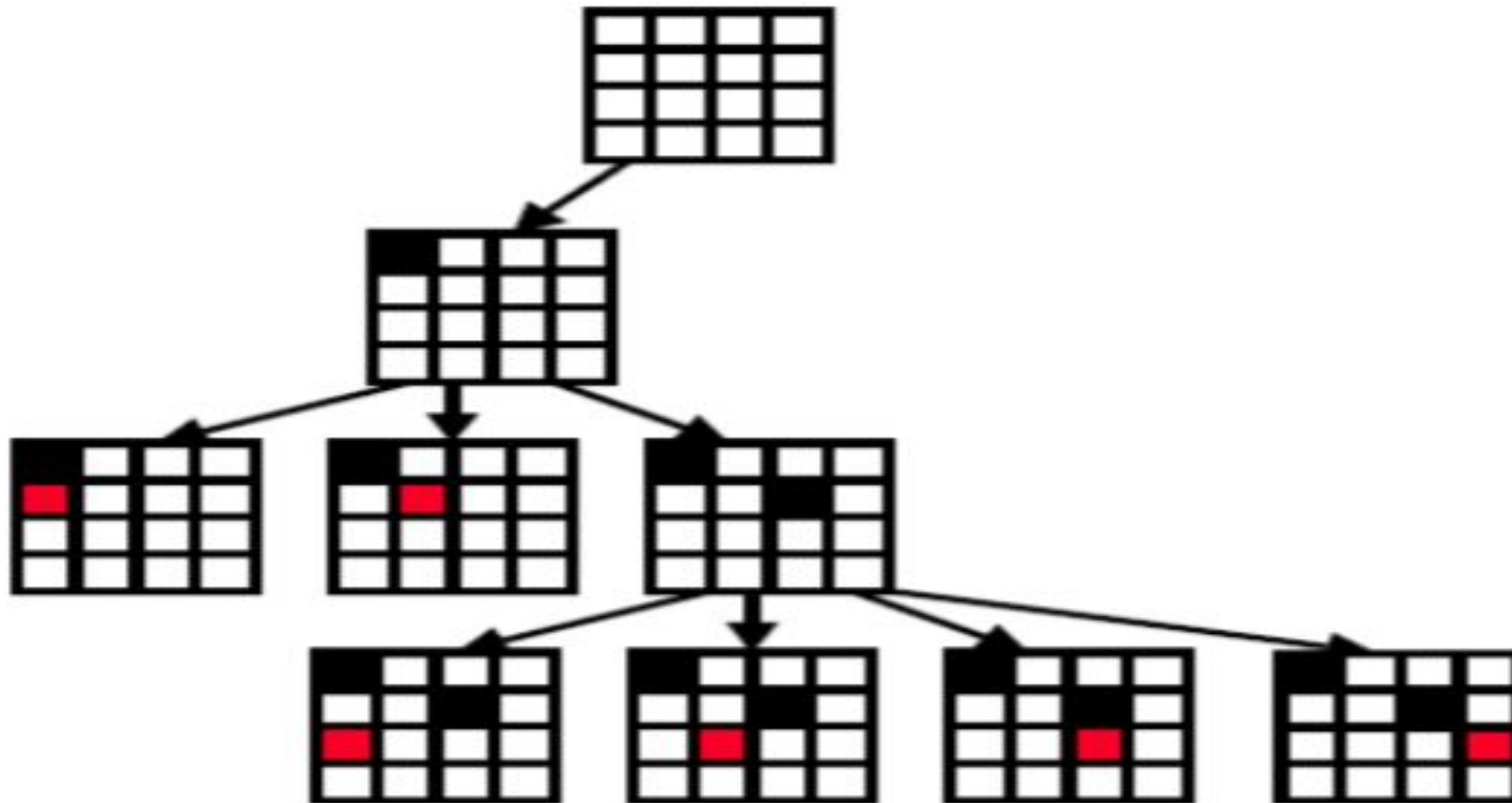


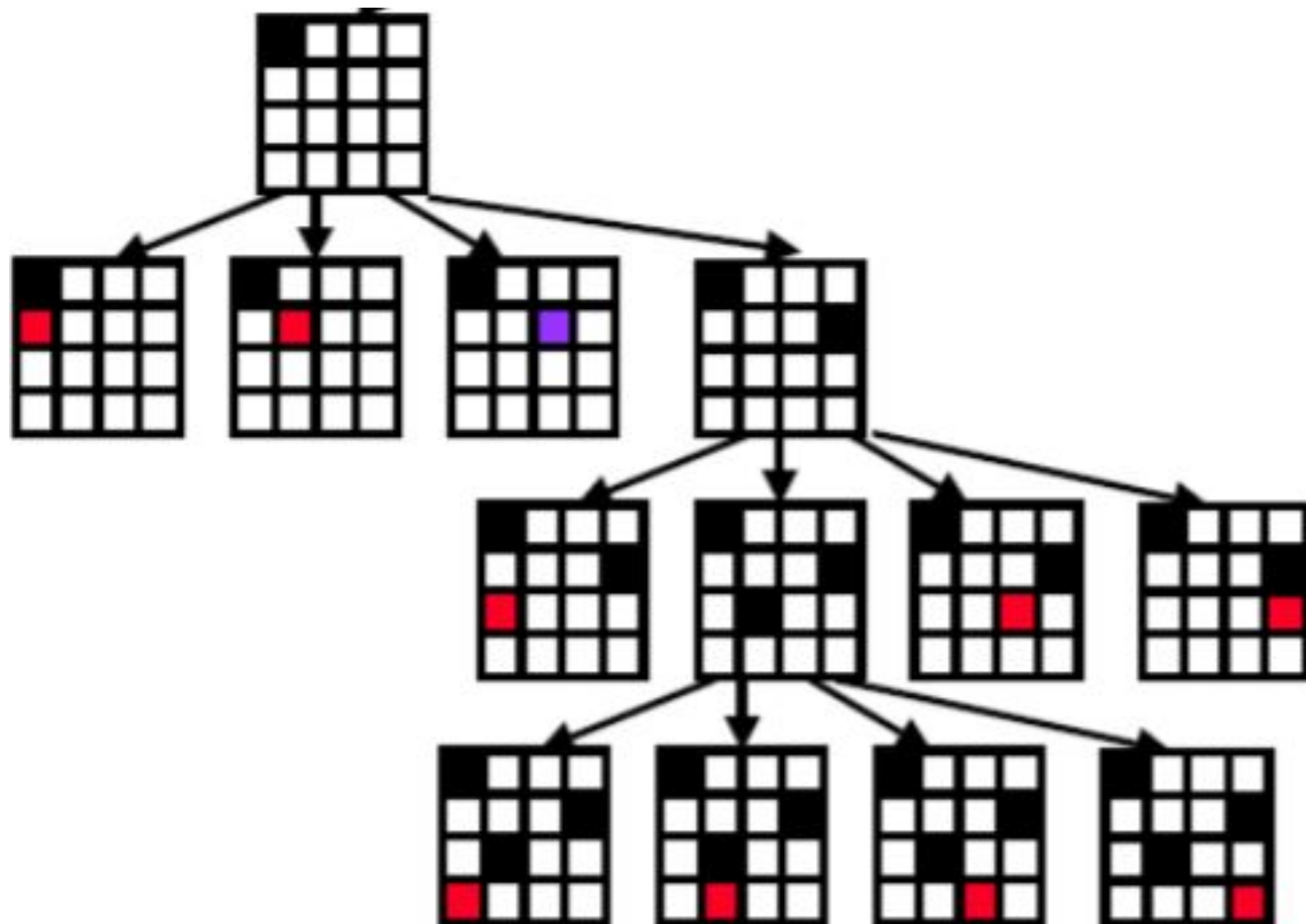
Intelligent Backtracking: Looking Backward

- Consider a fixed variable ordering **Q, NSW, V, T, SA, WA, NT**.
Suppose we have generated the partial assignment {Q=red, NSW =green, V =blue, T =red}.
When we try the next variable,
 - SA, we see that every value violates a constraint(chronological backtracking)
 - Recoloring Tasmania cannot possibly resolve the problem with South Australia.
 - A more intelligent approach to backtracking is to backtrack to a variable that might fix the problem
 - The set (in this case {Q=red ,NSW =green, V =blue, }), is called the **conflict set** for SA.
 - In this case, back jumping would jump over Tasmania and try a new value for V .

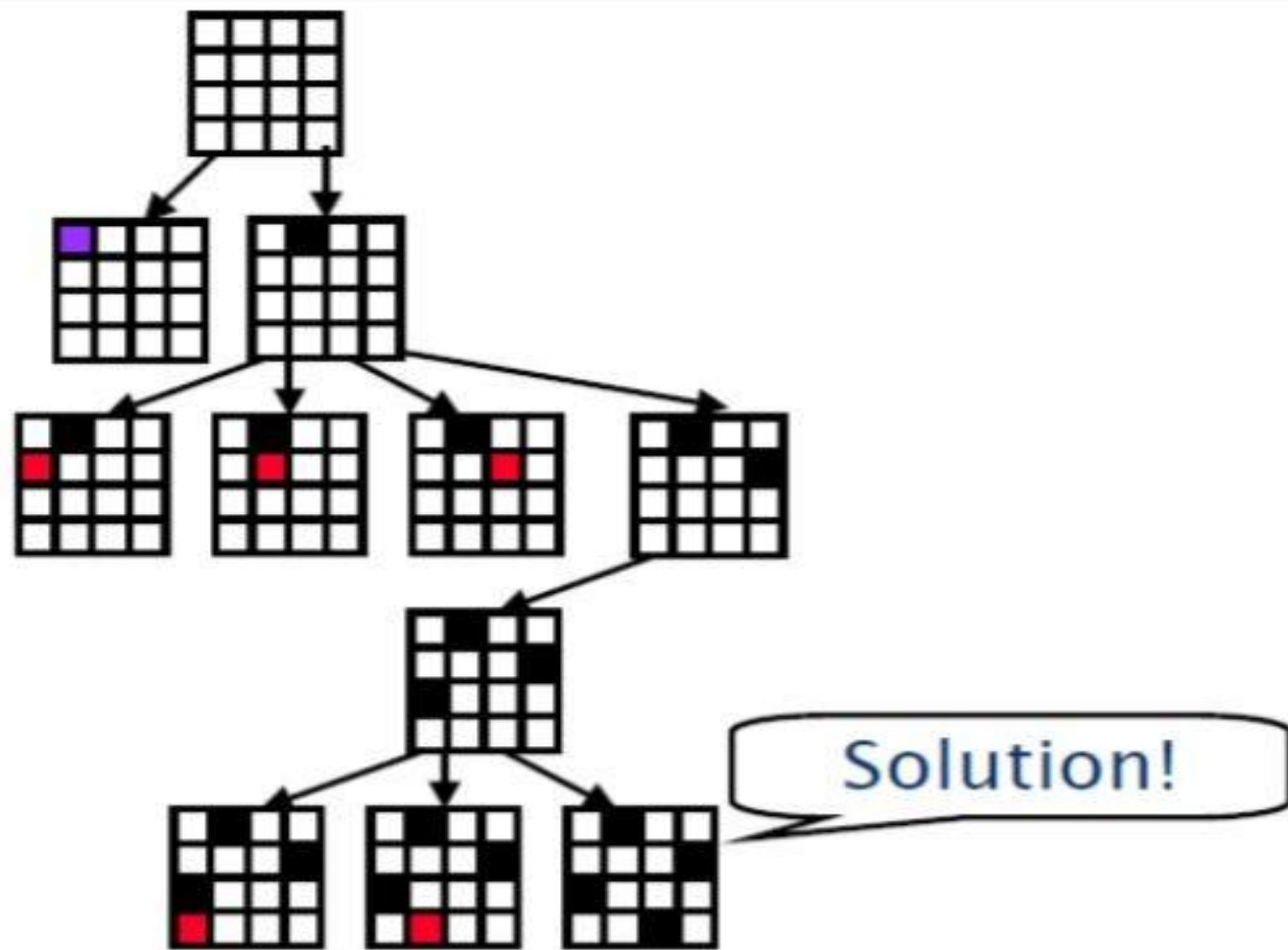
Backtracking Example:2

● 4X4 Queens





● 4X4 Queen



Back Tracking Example

Variables = { A, B, C }

Domain = { 1, 2, 3 }

Constraints = { $A > B$, B is not equal to C }

A = 1 □ B = 1 (Constraint Broken)

A = 1 □ B = 2 (Constraint Broken)

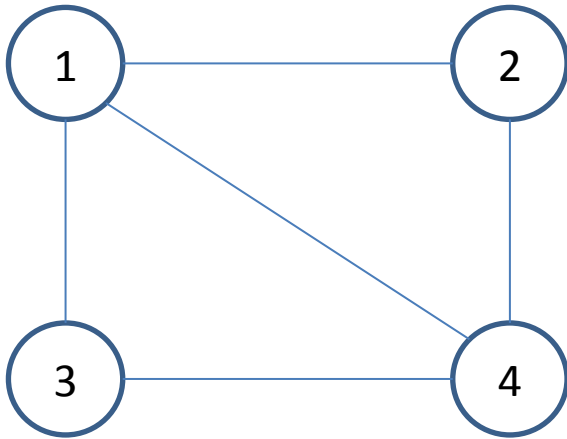
A = 1 □ B = 3 (Constraint Broken)

A = 2 □ B = 1 (Constraint Satisfied)

A=2 □ B = 1 □ C =1 (Con Violate)

A=2 B=1 C=2 (Con Satisfied)

CSP Algorithm Example



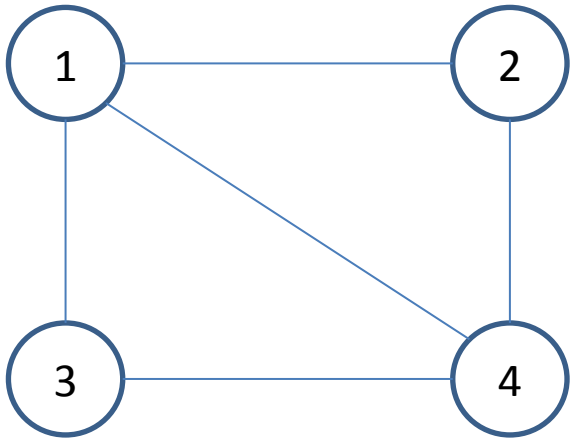
	1	2	3	4
Domain	RGB	RGB	RGB	RGB

$V = \{1, 2, 3, 4\}$

$D = \{\text{Red, Green, Blue}\}$

$C = \{1 \neq 2, 1 \neq 3, 1 \neq 4, 2 \neq 4, 3 \neq 4\}$

CSP Algorithm Example



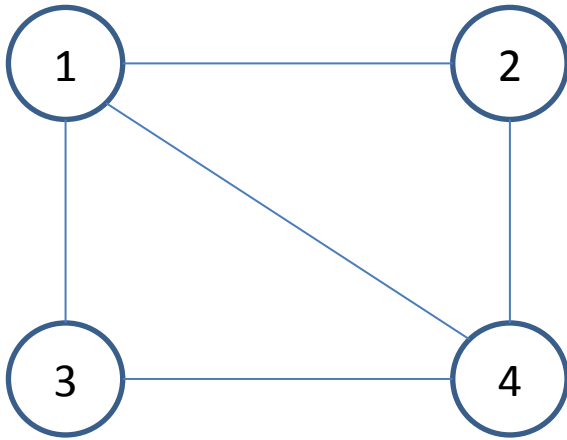
	1	2	3	4
Domain	RGB	RGB	RGB	RGB
1=R	R	GB	GB	GB

$V = \{1, 2, 3, 4\}$

$D = \{\text{Red, Green, Blue}\}$

$C = \{1 \neq 2, 1 \neq 3, 1 \neq 4, 2 \neq 4, 3 \neq 4\}$

CSP Algorithm Example



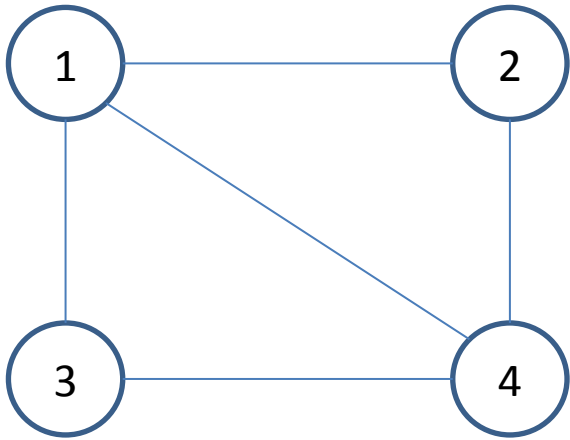
$V = \{1, 2, 3, 4\}$

$D = \{\text{Red, Green, Blue}\}$

$C = \{1 \neq 2, 1 \neq 3, 1 \neq 4, 2 \neq 4, 3 \neq 4\}$

	1	2	3	4
Domain	RGB	RGB	RGB	RGB
1=R	R	GB	GB	GB
2=G	R	G	GB	B

CSP Algorithm Example



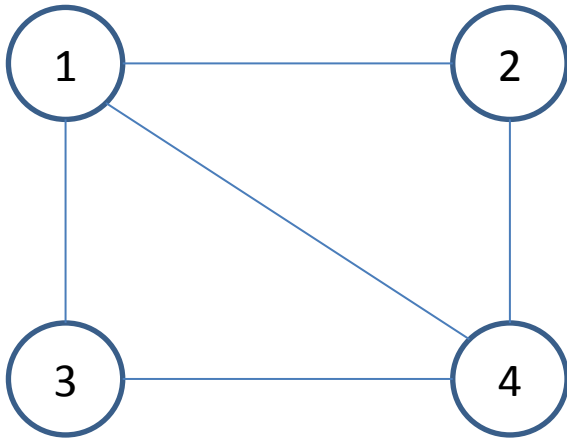
$V = \{1, 2, 3, 4\}$

$D = \{\text{Red, Green, Blue}\}$

$C = \{1 \neq 2, 1 \neq 3, 1 \neq 4, 2 \neq 4, 3 \neq 4\}$

	1	2	3	4
Domain	RGB	RGB	RGB	RGB
1=R	R	GB	GB	GB
2=G	R	G	GB	B
3=B	R	G	B	B

CSP Algorithm Example



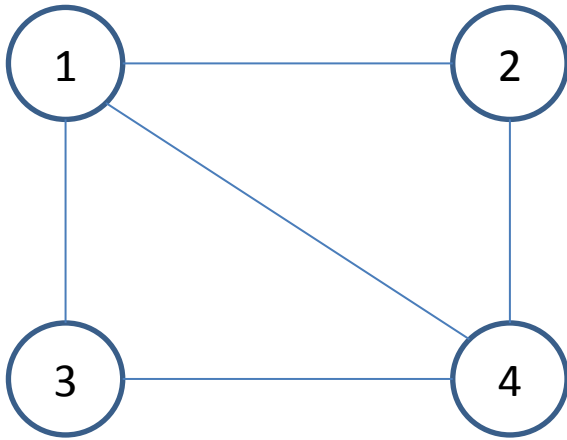
$V = \{1, 2, 3, 4\}$

$D = \{\text{Red, Green, Blue}\}$

$C = \{1 \neq 2, 1 \neq 3, 1 \neq 4, 2 \neq 4, 3 \neq 4\}$

	1	2	3	4
Domain	RGB	RGB	RGB	RGB
1=R	R	GB	GB	GB
2=G	R	G	GB	B
3=B	R	G	B	B
4=B	R	G	B	B

CSP Algorithm Example



$V = \{1, 2, 3, 4\}$

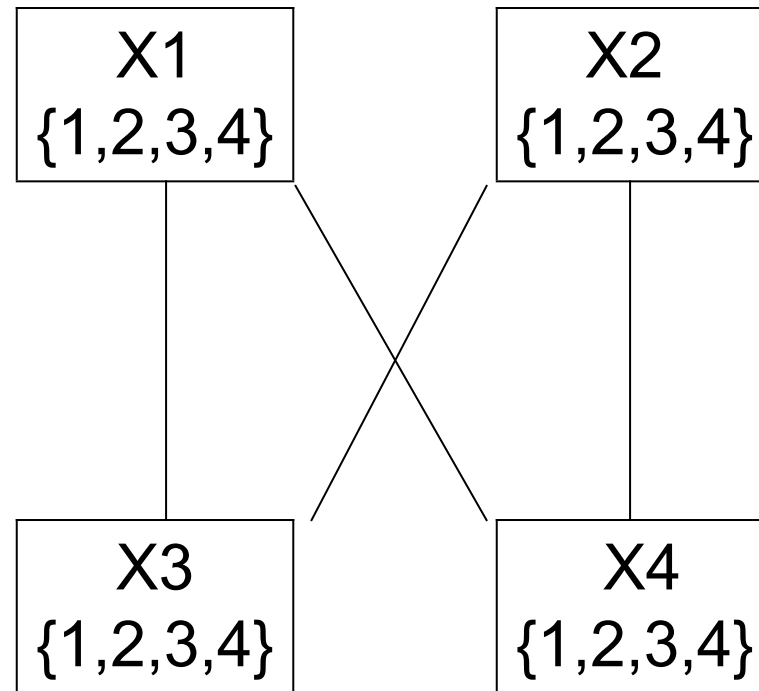
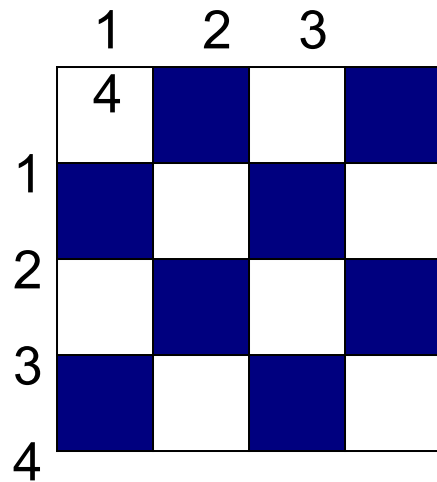
$D = \{\text{Red, Green, Blue}\}$

$C = \{1 \neq 2, 1 \neq 3, 1 \neq 4, 2 \neq 4, 3 \neq 4\}$

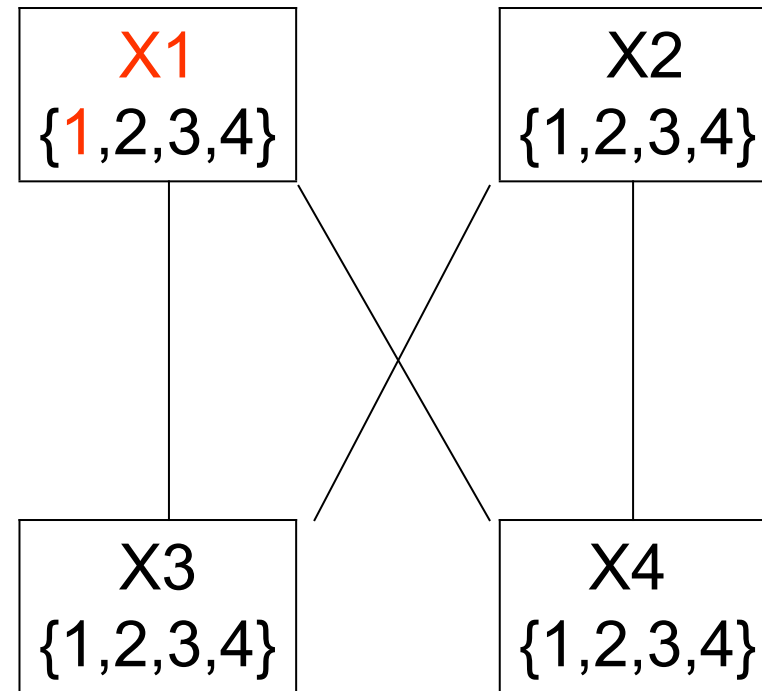
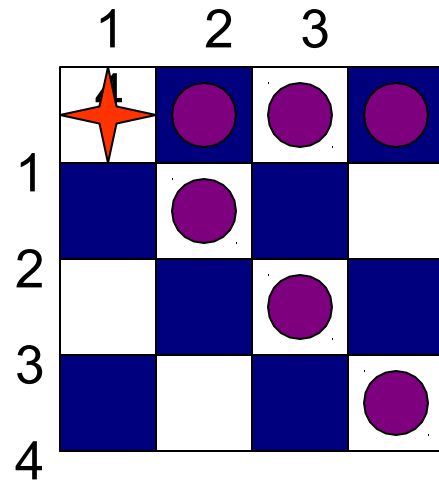


	1	2	3	4
Domain	RGB	RGB	RGB	RGB
1=R	R	GB	GB	GB
2=G	R	G	GB	B
3=G	R	G	G	B
4=B	R	G	G	B

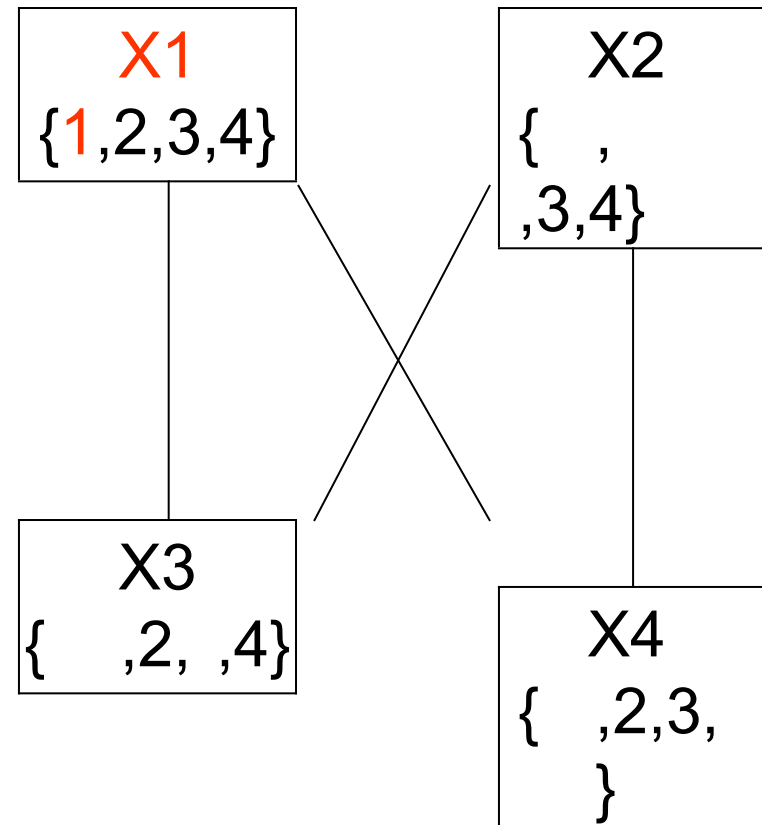
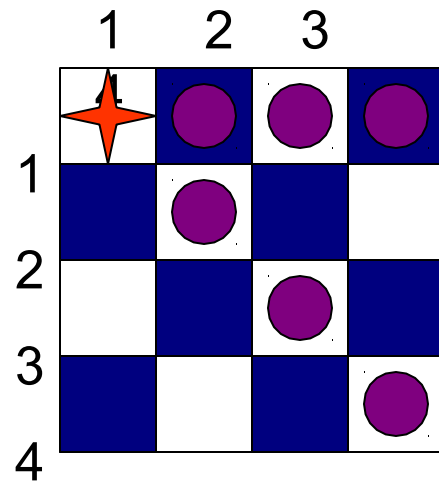
Example: 4-Queens Problem



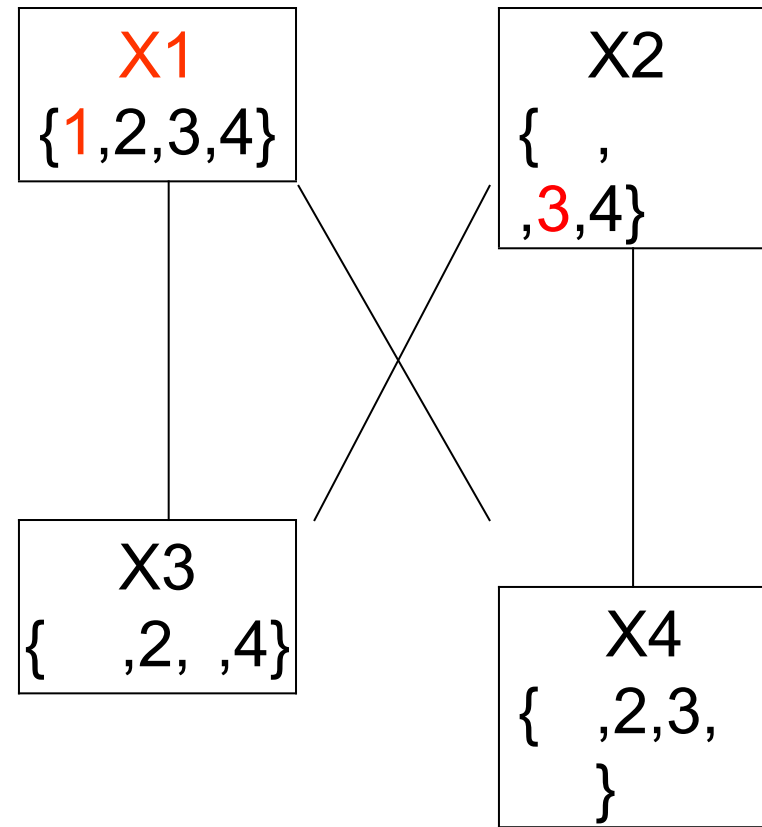
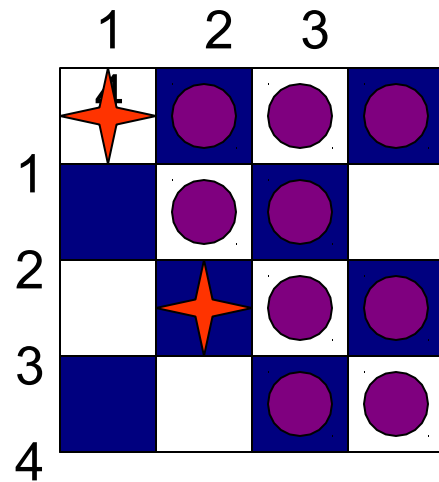
Example: 4-Queens Problem



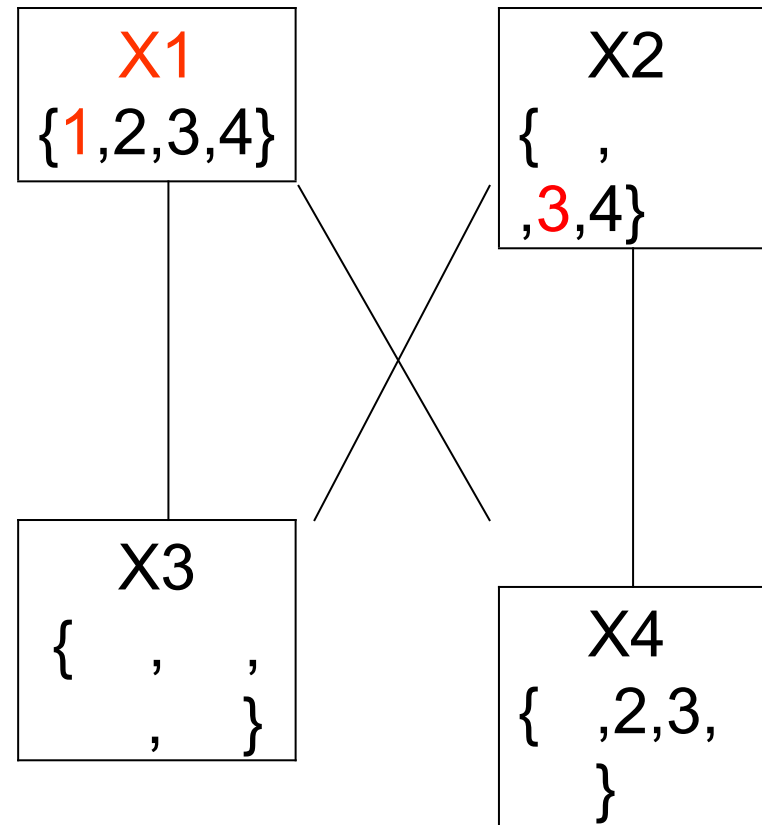
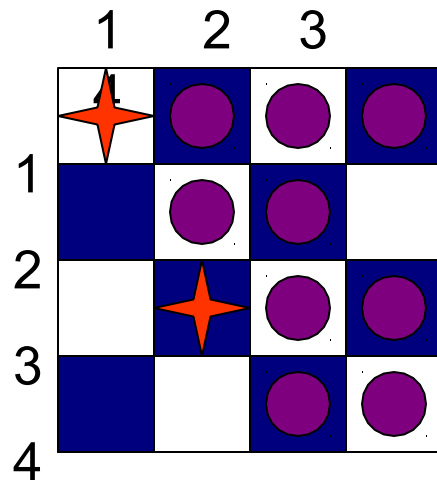
Example: 4-Queens Problem



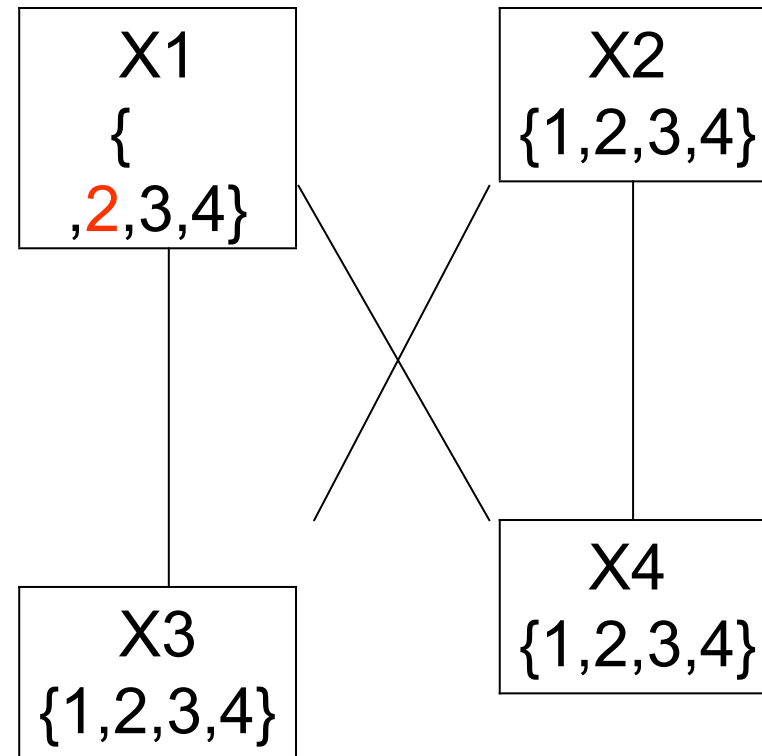
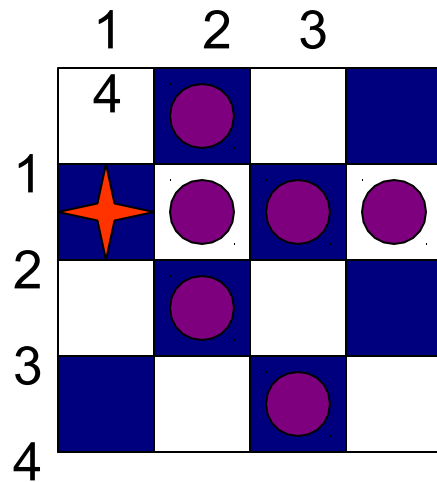
Example: 4-Queens Problem



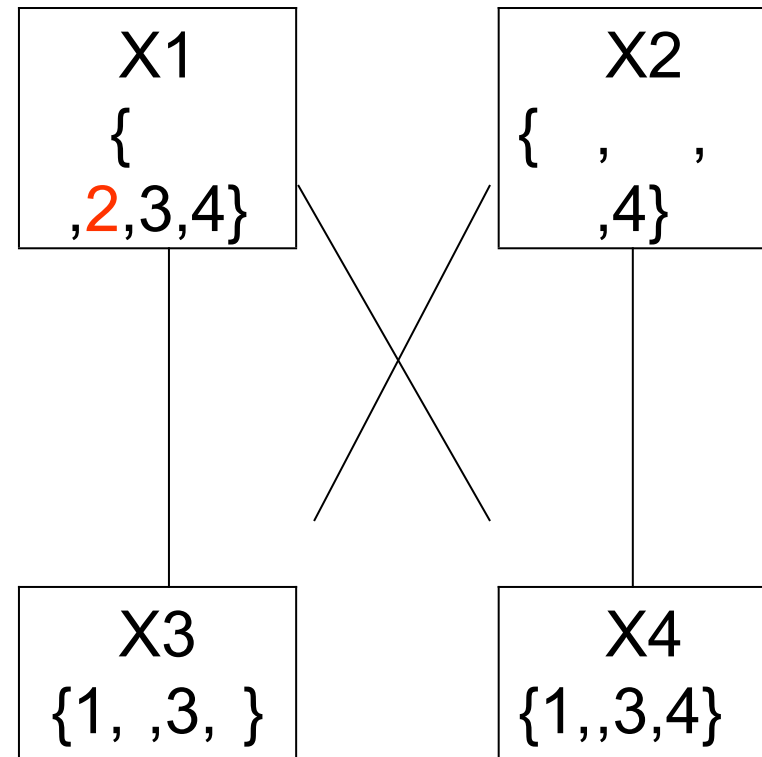
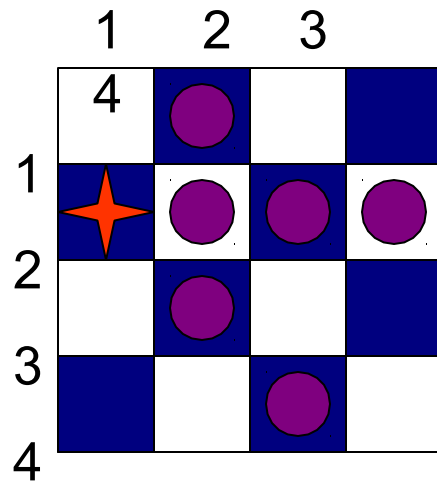
Example: 4-Queens Problem



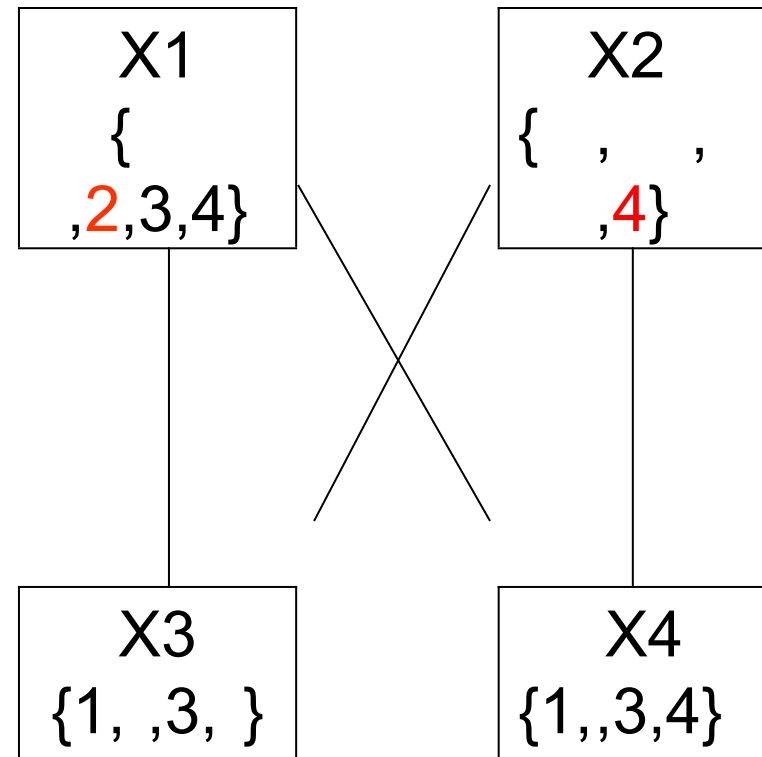
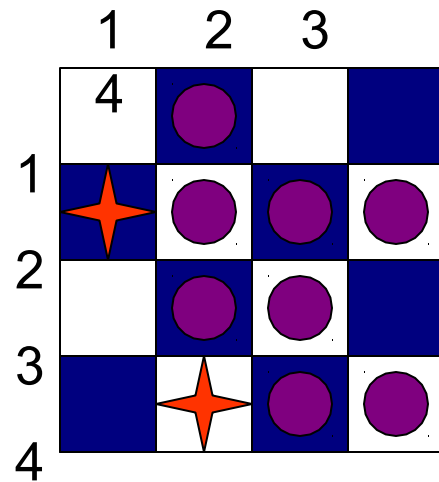
Example: 4-Queens Problem



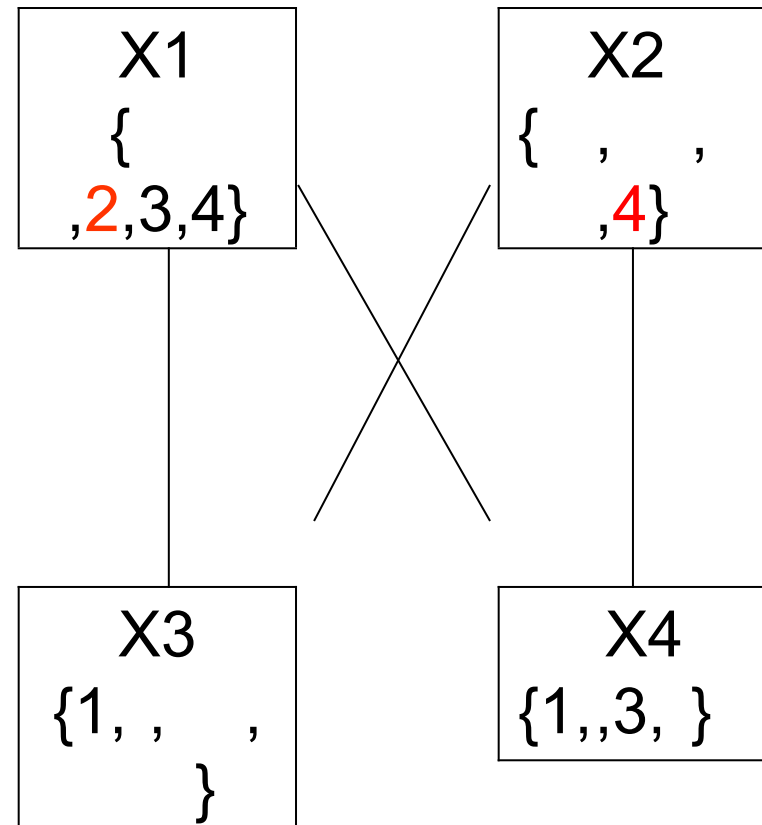
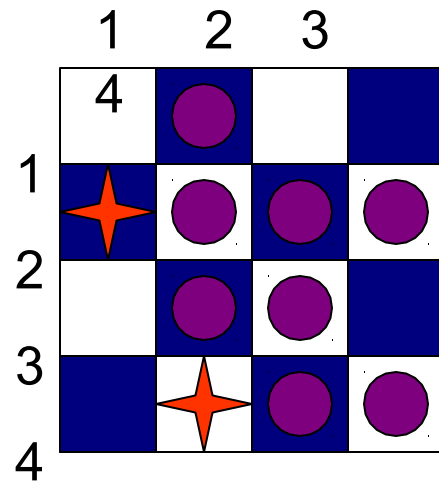
Example: 4-Queens Problem



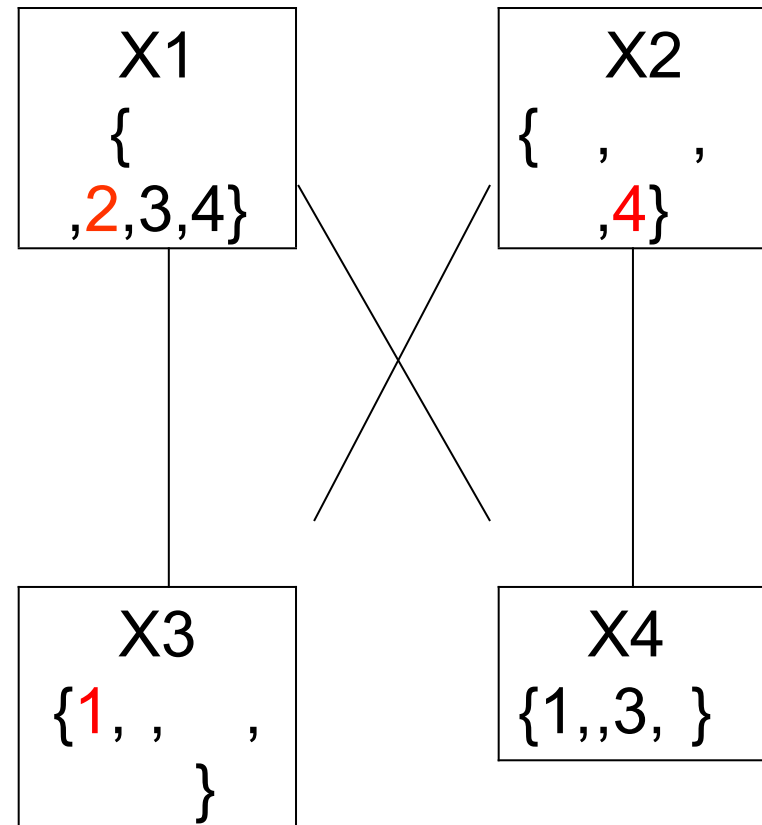
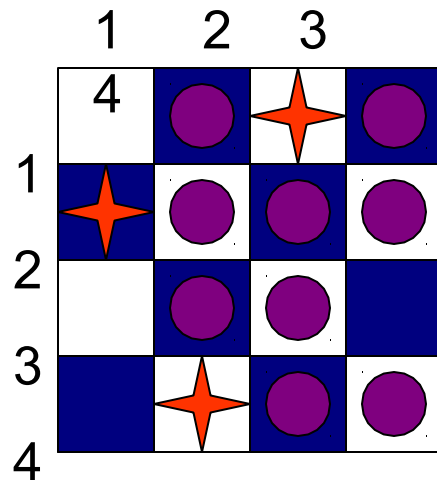
Example: 4-Queens Problem



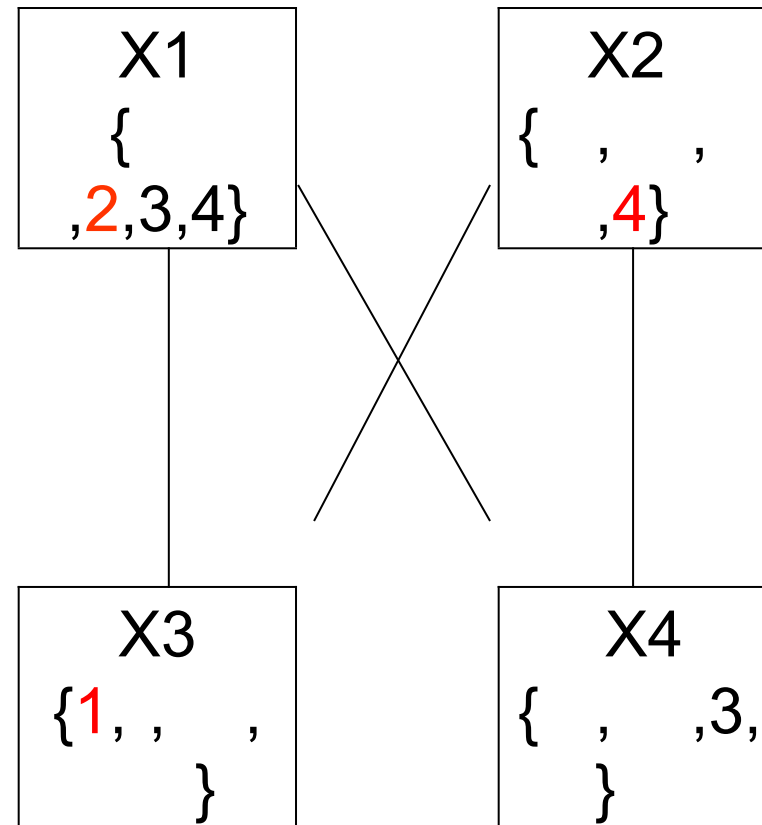
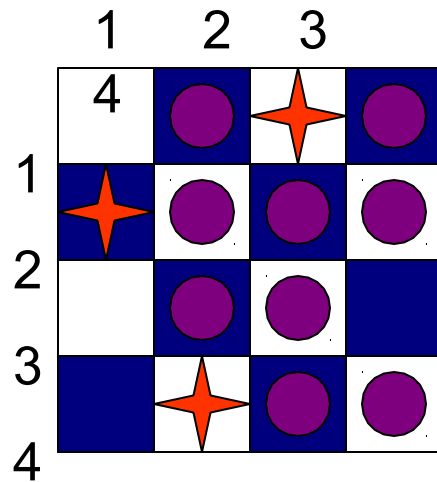
Example: 4-Queens Problem



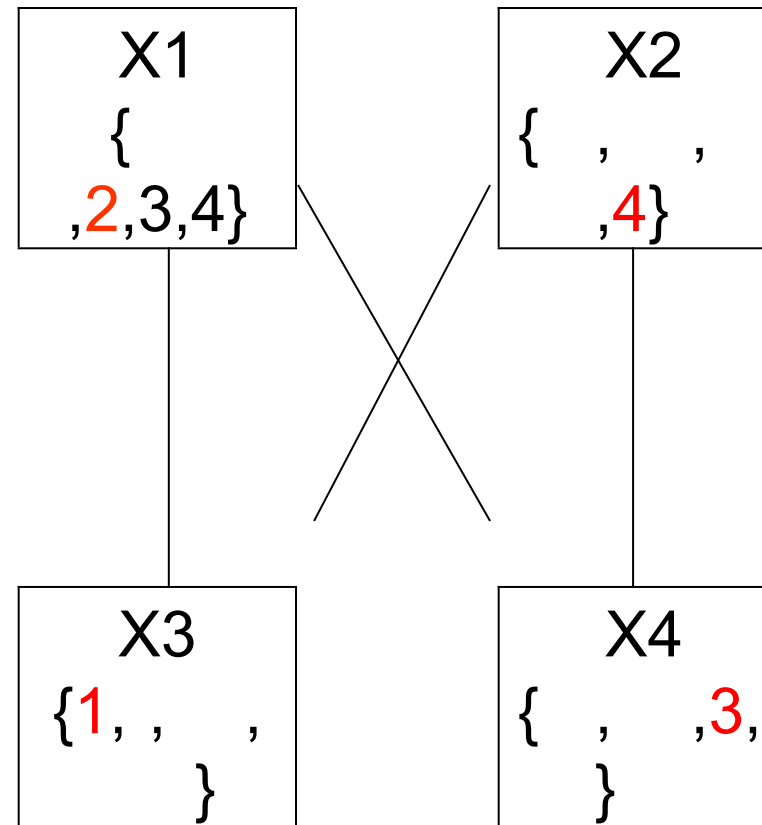
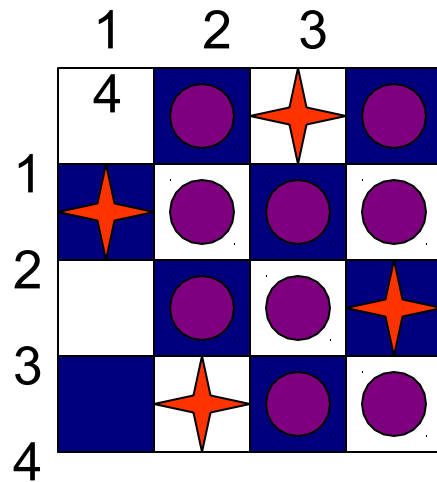
Example: 4-Queens Problem



Example: 4-Queens Problem



Example: 4-Queens Problem



Backtracking search

Backtracking = DFS + **variable-ordering** + **fail-on-violation**

Improving backtracking efficiency (Value Ordering and Filtering)

By default, Backtrack algorithm simply selects the next unassigned variable in the order given by the list `VARIABLES[csp]`.

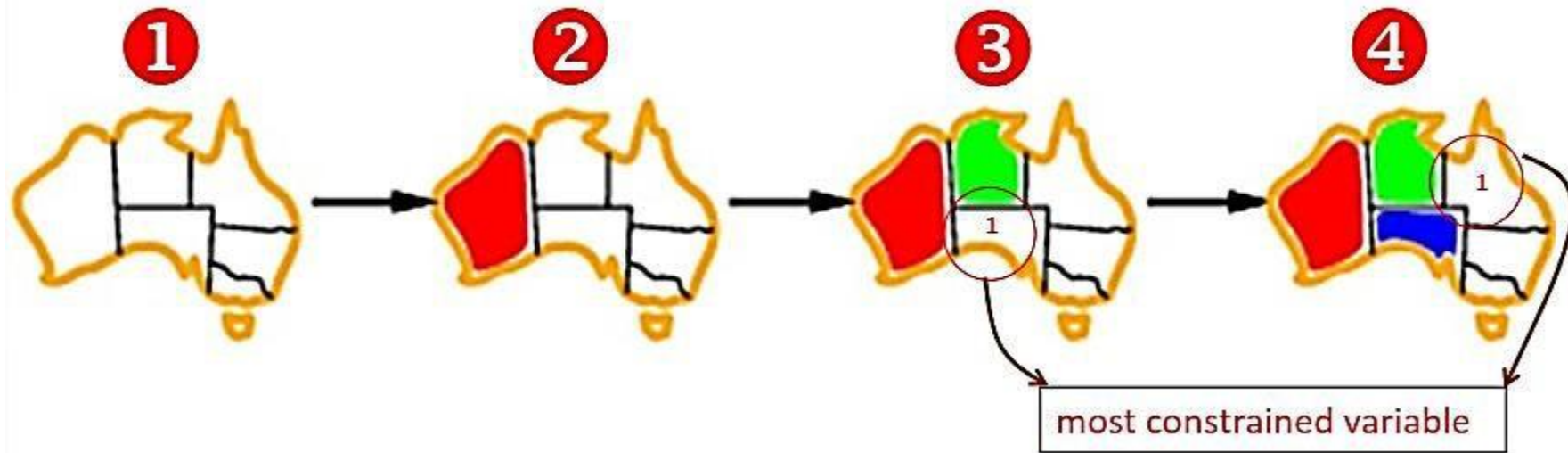
General-purpose methods can give huge gains in speed:

- Which variable should be assigned next?
- In what order should its values be tried ?
- Can we detect inevitable failure early?

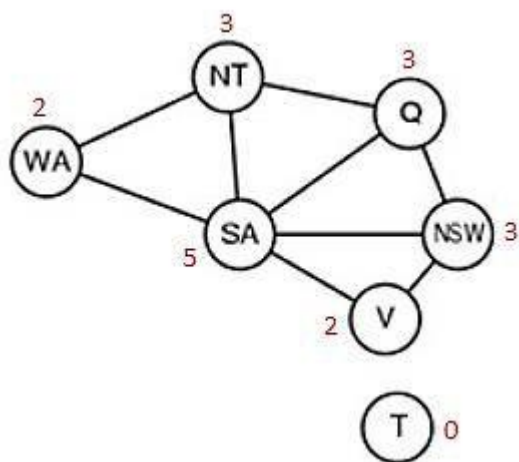
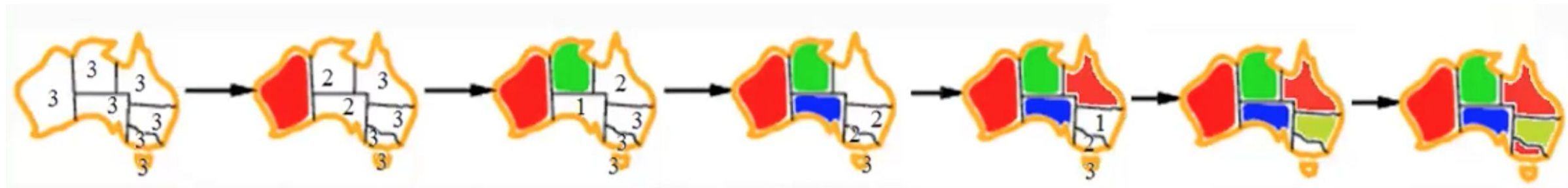
Ordering

HEURISTIC-1: Minimum remaining values (MRV)

- This intuitive idea-choosing the variable with the fewest "legal" values is called the minimum remaining values (MRV) heuristic (also known most constrained variable)



HEURISTIC-1: Minimum remaining values (MRV)



WA=(2)Red

NT(Remaining values)=2(Blue , Green)

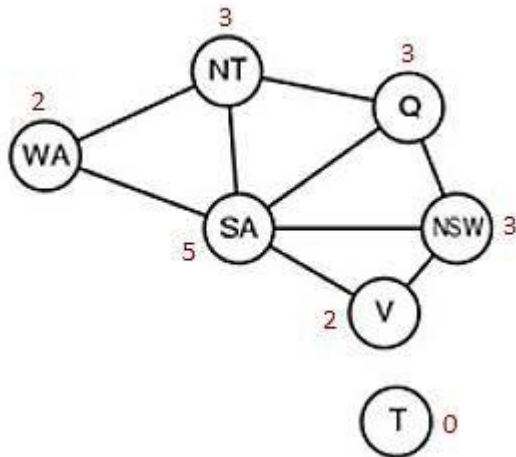
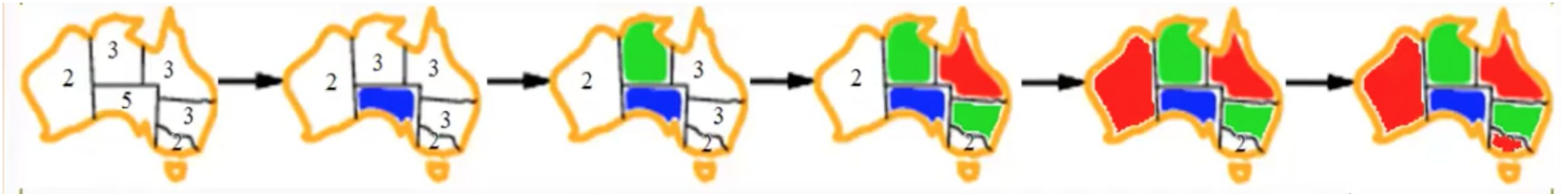
while Q and NSW have 3 remaining values so, go for NT

SA(Remaining values)= 1(Blue)

after the assignments for WA = red and NT = green, there is only one possible value for SA, so it makes sense to assign **SA = blue** next rather than assigning Q, after SA is assigned, the choices for Q, NSW, and V are all forced.

HEURISTIC-2: Most-Constraining Variable (Degree Heuristics)

- Most constraining variable: choose the variable with the most constraints on remaining variables

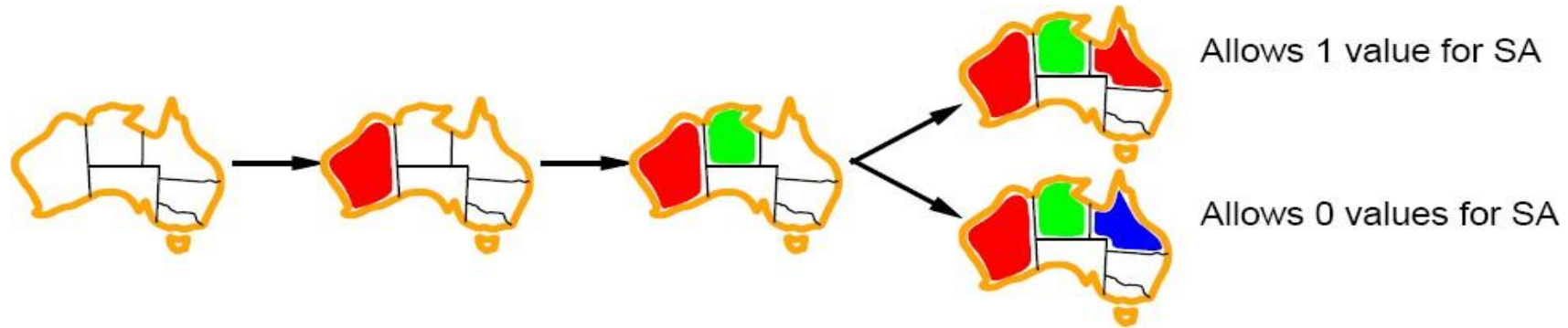


SA=5 (Most Constraints) BLUE

NT, Q, NSW = 3 **TIE** b/w them break it by choosing the first choice either NT or NSW.

So, choose NT Then Q, NSW OR Q, Then Q and NT

HEURISTIC-3: Least constraining value



- Least constraining value heuristic
- Rule: given a variable choose the least constraining value i.e. the one that leaves the maximum flexibility for subsequent variable assignments.

Filtering

HEURISTIC-4: Forward Checking

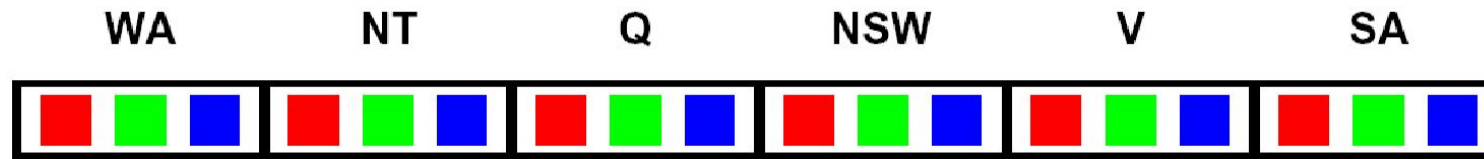
Filtering: Keep track of domains for unassigned variables and cross off bad options

Forward checking: Cross off values that violate a constraint when added to the existing assignment

Filtering: Forward Checking

Filtering: Keep track of domains for unassigned variables and cross off bad options

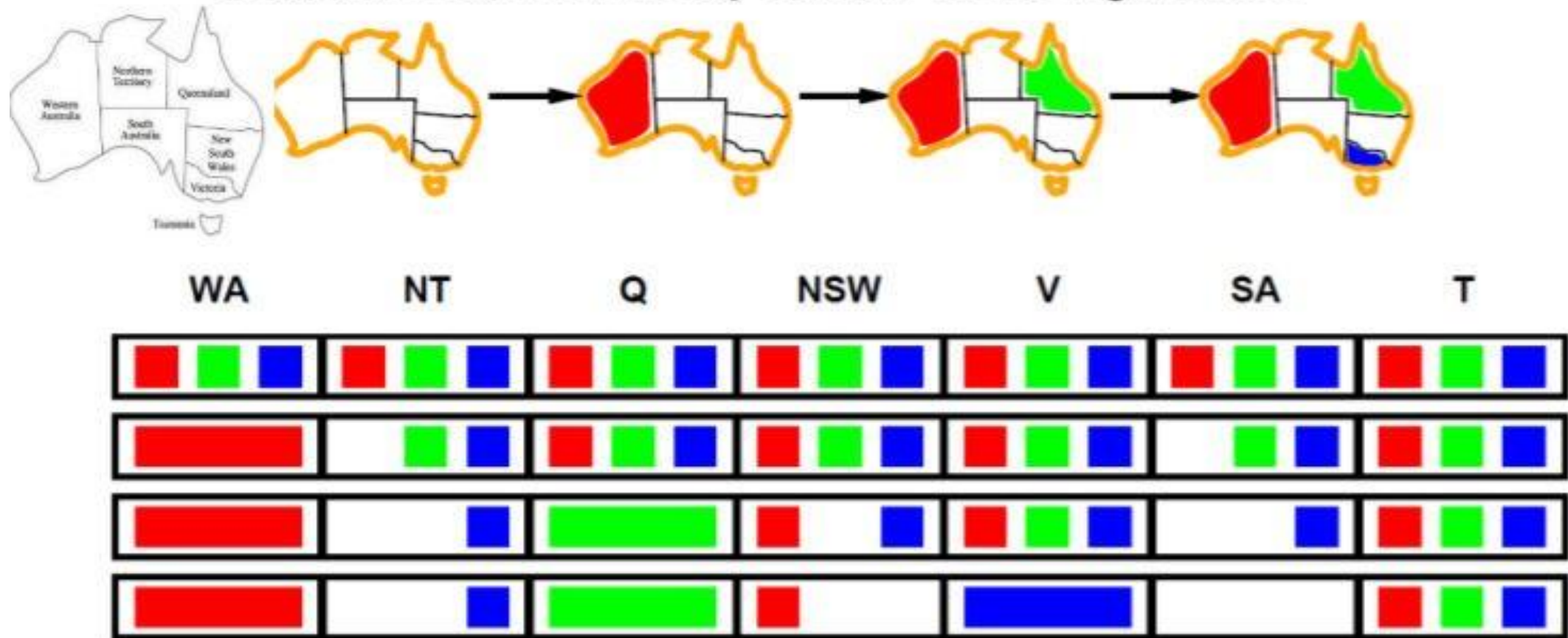
Forward checking: Cross off values that violate a constraint when added to the existing assignment



Forward Checking

Idea: Keep track of remaining legal values for unassigned variables

Terminate search when any variable has no legal values

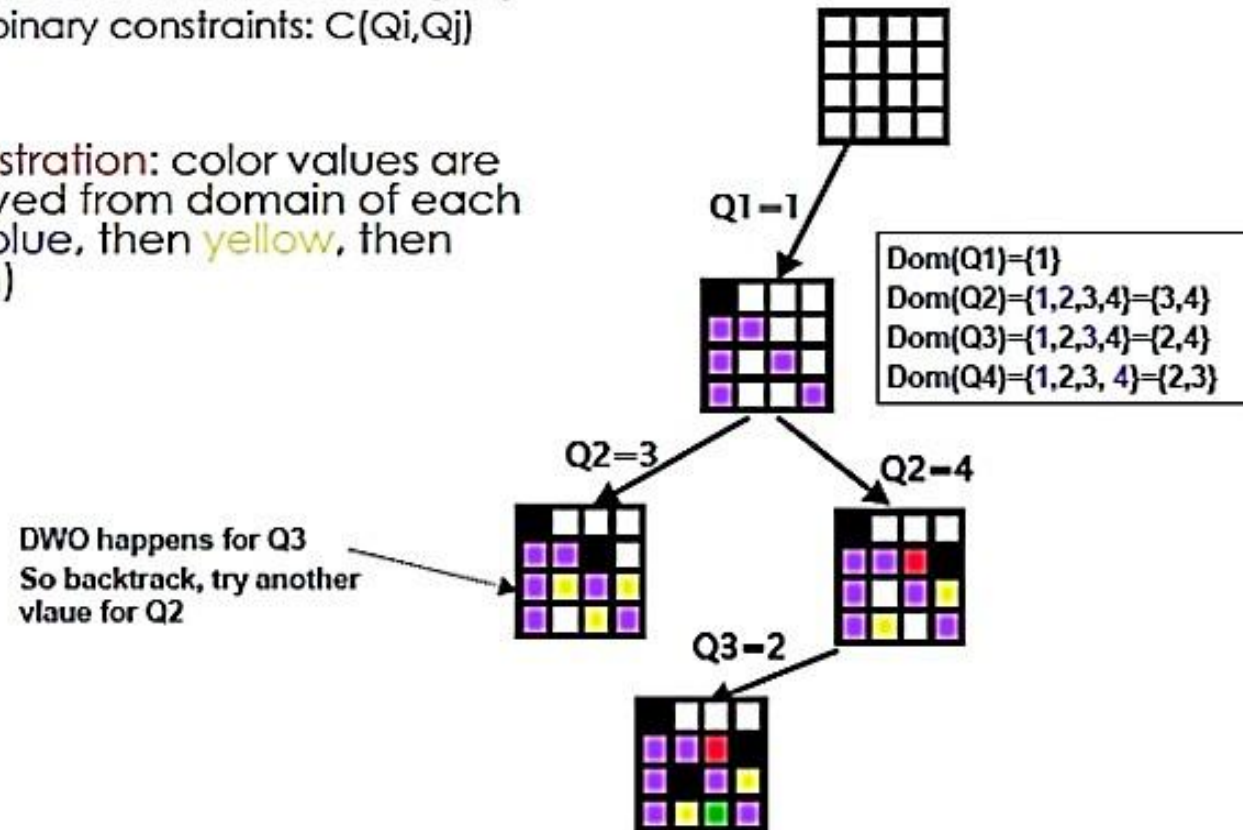


No possible assignments for SA, we try other assignments

Forward Checking Example:2

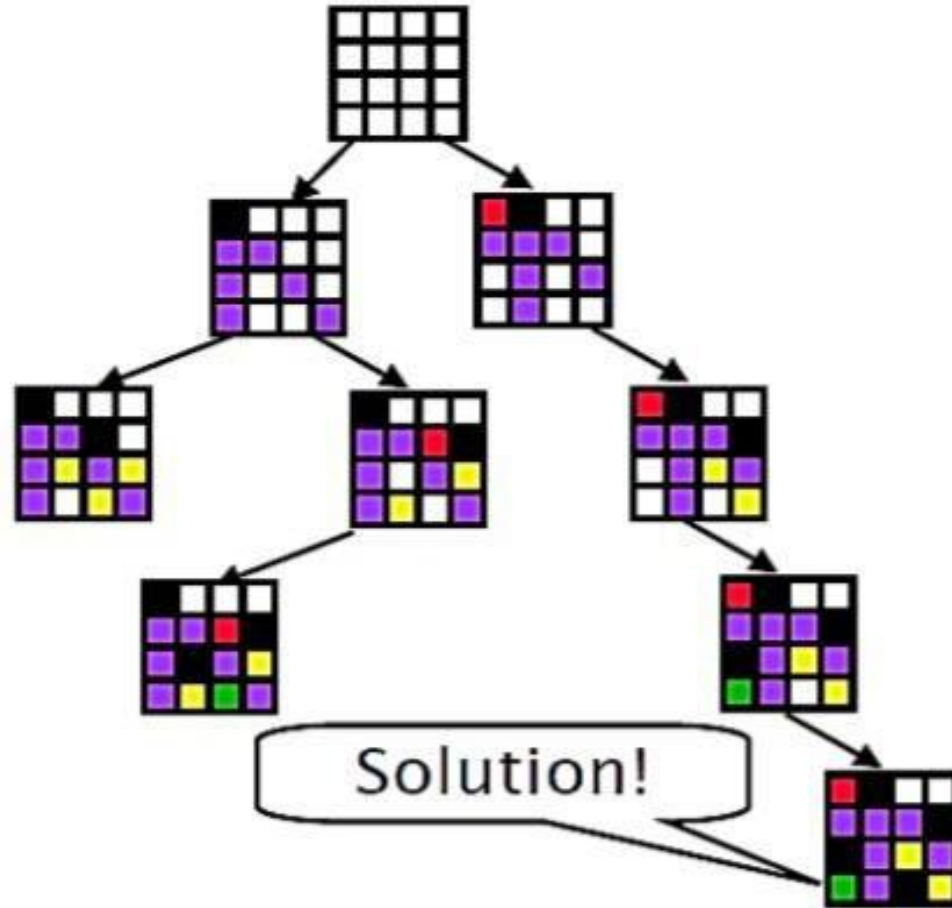
- 4X4 Queens
 - Q1,Q2,Q3,Q4 with domain {1..4}
 - All binary constraints: $C(Q_i, Q_j)$

- FC illustration: color values are removed from domain of each row (blue, then yellow, then green)

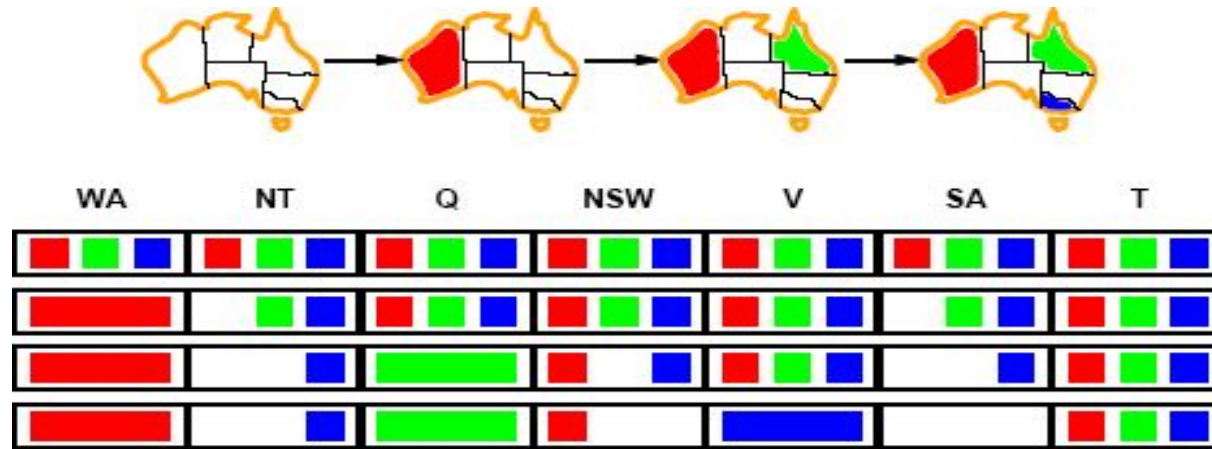


Forward Checking Example:2.....

- 4X4 Queens continue...



Constraint propagation

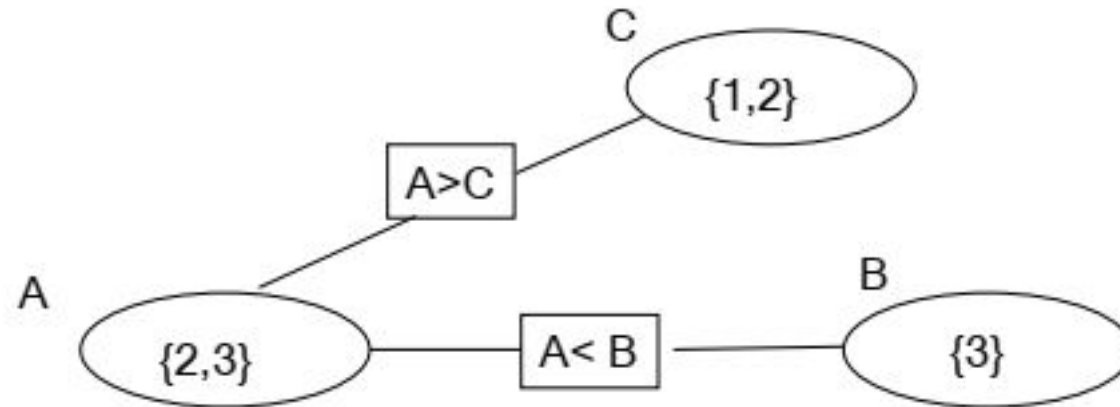


- Solving CSPs with combination of heuristics plus forward checking is more efficient than either approach alone.
- FC checking propagates information from assigned to unassigned variables but does not provide detection for all failures.
 - **NT and SA cannot be blue!**
- Constraint propagation repeatedly enforces constraints locally

Constraint Networks

Def. A **constraint network** is defined by a graph, with

- one **node** for every **variable** (drawn as **circle**)
- one **node** for every **constraint** (drawn as **rectangle**)
- **undirected edges** running between **variable nodes** and **constraint nodes** whenever a given variable is involved in a given constraint.



- Three variables: A, B, C
- Two constraints: A < B,
A > C

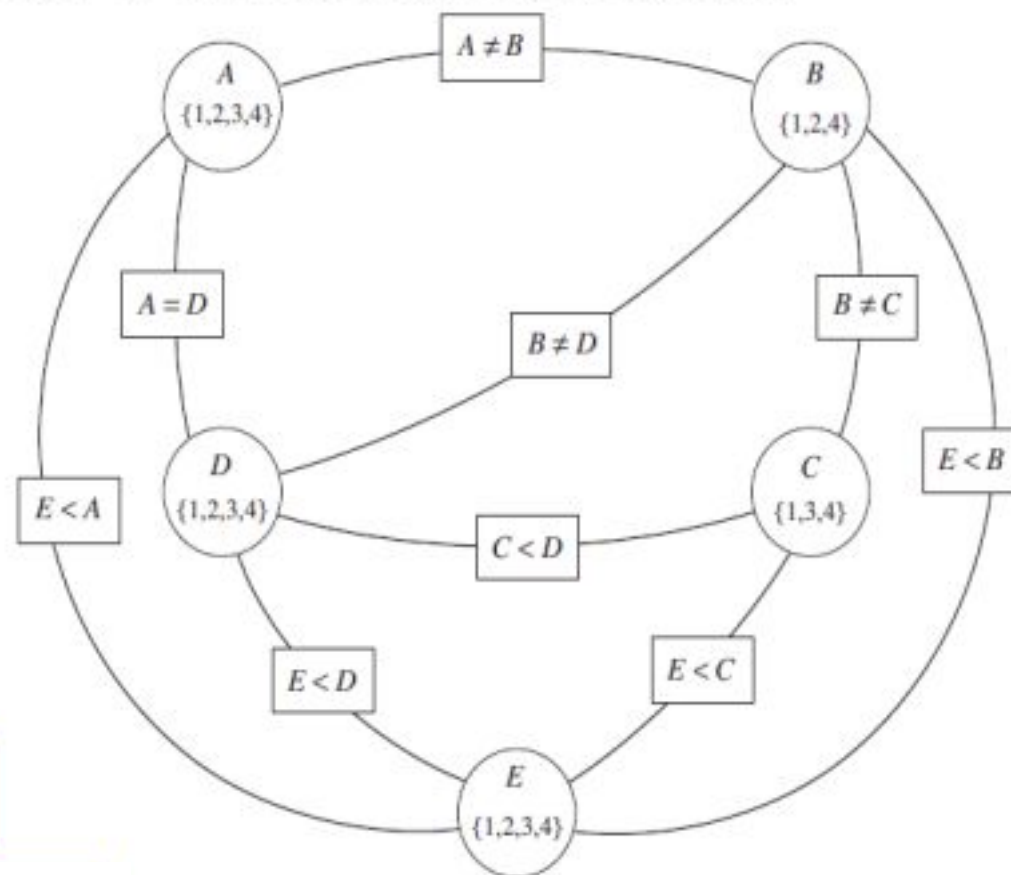
A more complicated example

How many variables are there in this constraint network?

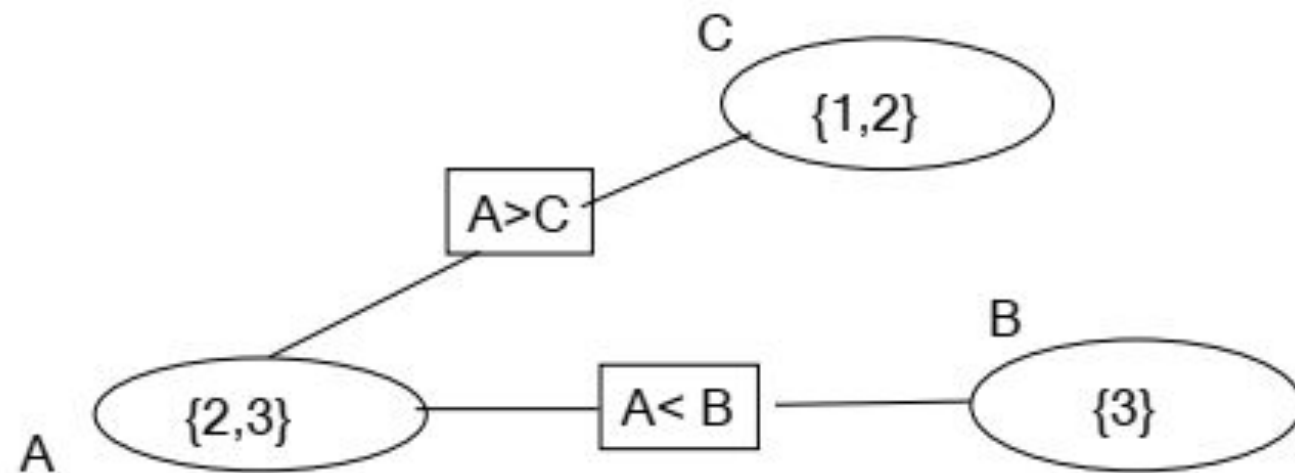
5

How many constraints?

9



Arc Consistency



Arc Consistency

AC-3 Algorithm Steps:

1) Turn each binary constraint into two arcs e.g.

$A > B$ will become $A > B$, $B < A$

2) Add all arcs to a **Queue**, also called as **Agenda**

3) Repeat Until Queue/Agenda is empty

Take an arc (**X_i** , **X_j**) from the agenda and check it

For every value of **X_i** there should be value of **X_j**

Remove/prune any inconsistent values from **X_i**

If **X_i** has **changed**, add all arcs of the form (**X_i** , **X_j**) to the

Arc Consistency

Variables = { A, B, C }

A = { 1, 2, 3 }

Domain = { 1, 2, 3 }

B = { 1, 2, 3 }

Constraints = { A > B, B = C }

C = { 1, 2, 3 }

STEP 1: Turn each binary constraint into two arcs.

A > B $\square$ B < A

B = C $\square$ C = B

Arc Consistency

Variables = { A, B, C }

A = { 1, 2, 3 }

Domain = { 1, 2, 3 }

B = { 1, 2, 3 }

Constraints = { A > B, B = C }

C = { 1, 2, 3 }

STEP 1: Turn each binary constraint into two arcs.

A > B □ B < A

B = C □ C = B

STEP 2: Add all arcs to the agenda.

Agenda:

A > B

B < A

B = C

C = B

Arc Consistency

Variables = { A, B, C }

A = { 2, 3 }

Domain = { 1, 2, 3 }

B = { 1, 2, 3 }

Constraints = { A > B, B = C }

C = { 1, 2, 3 }

STEP 1: Turn each binary constraint into two arcs.

A > B $\square$ B < A

B = C $\square$ C = B

STEP 3: If X_i is changed Check (X_j , X_i)

STEP 2: Add all arcs to the agenda.

Agenda:

~~A > B~~

B < A

B = C

C = B

Arc Consistency

Variables = { A, B, C }

A = { 2, 3 }

Domain = { 1, 2, 3 }

B = { 1, 2, 3 }

Constraints = { A > B, B = C }

C = { 1, 2, 3 }

STEP 1: Turn each binary constraint into two arcs.

A > B $\square$ B < A

B = C $\square$ C = B

STEP 3: If X_i is changed Check (X_j , X_i)

STEP 2: Add all arcs to the agenda.

Agenda:

~~A > B~~

~~B < A~~

B = C

C = B

Arc Consistency

Variables = { A, B, C }

A = { 2, 3 }

Domain = { 1, 2, 3 }

B = { 1, 2 }

Constraints = { A > B, B = C }

C = { 1, 2, 3 }

STEP 1: Turn each binary constraint into two arcs.

A > B $\square$ B < A

B = C $\square$ C = B

STEP 3: If X_i is changed Check (X_j , X_i)

STEP 2: Add all arcs to the agenda.

Agenda:

~~A > B~~

~~B < A~~

B = C

C = B

A > B

Arc Consistency

Variables = { A, B, C }

A = { 2, 3 }

Domain = { 1, 2, 3 }

B = { 1, 2 }

Constraints = { A > B, B = C }

C = { 1, 2, 3 }

STEP 1: Turn each binary constraint into two arcs.

A > B $\square$ B < A

B = C $\square$ C = B

STEP 3: If X_i is changed Check (X_j , X_i)

STEP 2: Add all arcs to the agenda.

Agenda:

~~A > B~~

~~B < A~~

~~B = C~~

C = B

A > B

Arc Consistency

Variables = { A, B, C }

A = { 2, 3 }

Domain = { 1, 2, 3 }

B = { 1, 2 }

Constraints = { A > B, B = C }

C = { 1, 2 }

STEP 1: Turn each binary constraint into two arcs.

A > B $\square$ B < A

B = C $\square$ C = B

STEP 3: If X_i is changed Check (X_j , X_i)

STEP 2: Add all arcs to the agenda.

Agenda:

~~A > B~~

~~B < A~~

~~B = C~~

~~C = B~~

A > B

B = C

Arc Consistency

Variables = { A, B, C }

A = { 2, 3 }

Domain = { 1, 2, 3 }

B = { 1, 2 }

Constraints = { A > B, B = C }

C = { 1, 2 }

STEP 1: Turn each binary constraint into two arcs.

A > B $\square$ B < A

B = C $\square$ C = B

STEP 3: If X_i is changed Check (X_j , X_i)

STEP 2: Add all arcs to the agenda.

Agenda:

~~A > B~~

~~B < A~~

~~B = C~~

~~C = B~~

~~A > B~~

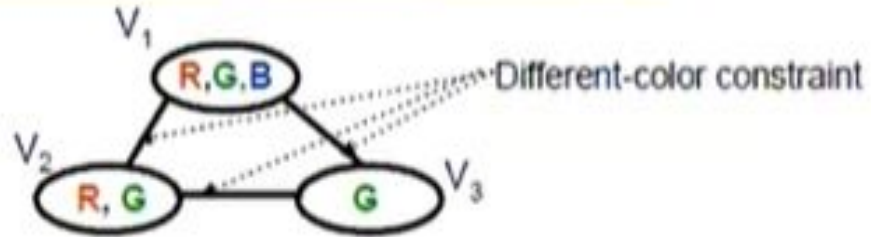
~~B = C~~

ARC Consistency

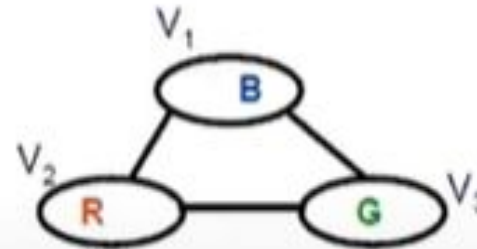
Constraint Propagation Example

Graph Coloring

Initial Domains are indicated

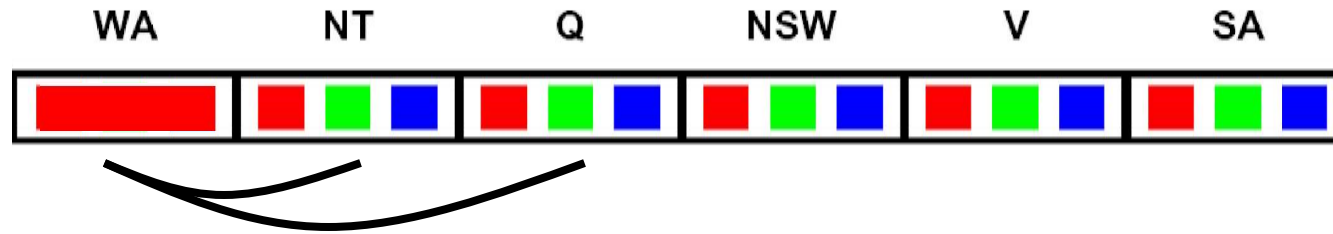
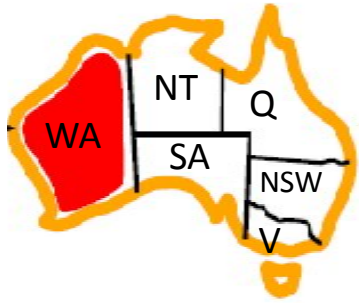


Arc examined	Value deleted
$V_1 - V_2$	none
$V_1 - V_3$	$V_1(G)$
$V_2 - V_3$	$V_2(G)$
$V_1 - V_2$	$V_1(R)$
$V_1 - V_3$	none
$V_2 - V_3$	none



Consistency of A Single Arc

An arc $X \rightarrow Y$ is consistent iff for every x in the tail there is some y in the head which could be assigned without violating a constraint

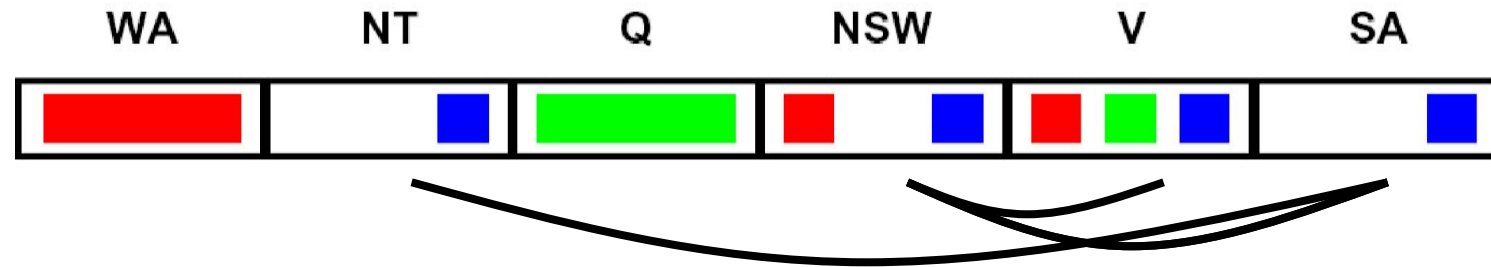
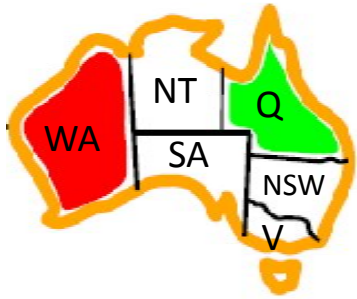


Forward checking: Enforcing consistency of arcs pointing to each new assignment

Delete from the tail!

Arc Consistency of an Entire CSP

A simple form of propagation makes sure all arcs are consistent:



Important: If X loses a value, neighbors of X need to be rechecked!

Arc consistency detects failure earlier than forward checking

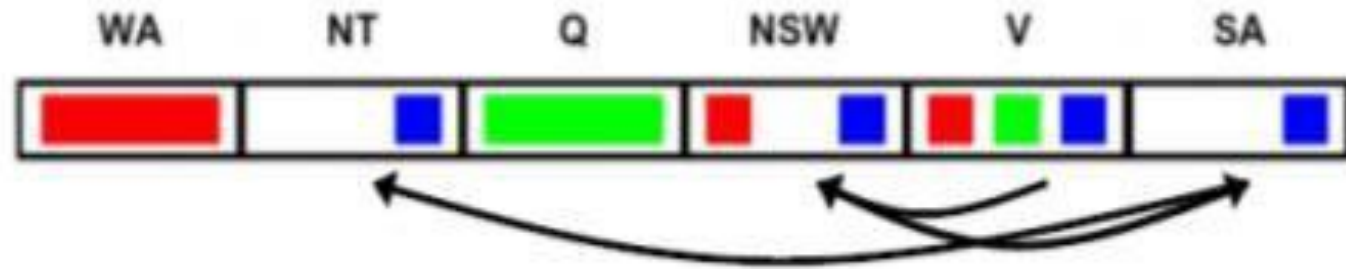
Can be run as a preprocessor or after each assignment

What's the downside of enforcing arc consistency?

*Remember:
Delete from
the tail!*

ARC Consistency

- A simple form of propagation makes sure **all** arcs are simultaneously consistent:



- Arc consistency detects failure earlier than forward checking
- Important: If X loses a value, neighbors of X need to be rechecked!
- Must rerun after each assignment!

*Remember:
Delete from
the tail!*

function AC-3(*csp*) **returns** the CSP, possibly with reduced domains

inputs: *csp*, a binary CSP with variables $\{X_1, X_2, \dots, X_n\}$

local variables: *queue*, a queue of arcs, initially all the arcs in *csp*

while *queue* is not empty **do**

$(X_i, X_j) \leftarrow \text{REMOVE-FIRST}(\textit{queue})$

if REMOVE-INCONSISTENT-VALUES(X_i, X_j) **then**

for each X_k **in** NEIGHBORS[X_i] **do**

 add (X_k, X_i) to *queue*

function REMOVE-INCONSISTENT-VALUES(X_i, X_j) **returns** true iff succeeds

removed $\leftarrow$ false

for each x **in** DOMAIN[X_i] **do**

if no value y in DOMAIN[X_j] allows (x, y) to satisfy the constraint $X_i \leftrightarrow X_j$

then delete x from DOMAIN[X_i]; *removed* $\leftarrow$ true

return *removed*

Limitations of Arc Consistency

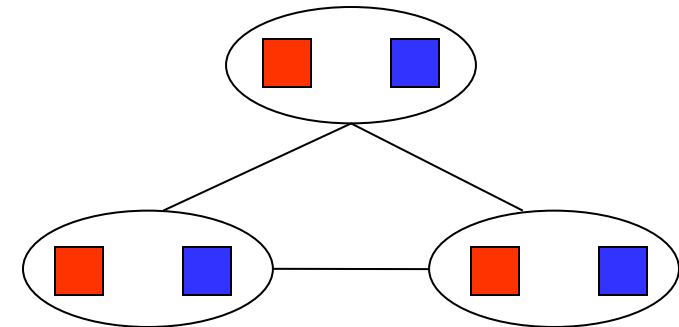
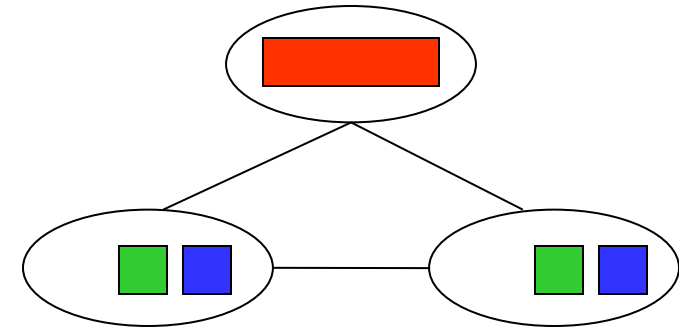
After enforcing arc consistency:

- Can have one solution left

- Can have multiple solutions left

- Can have no solutions left (and not know it)

Arc consistency still runs inside a backtracking search!



*What went
wrong here?*

Summary

- CSPs are a special kind of problem: states defined by values of a fixed set of variables, goal test defined by constraints on variable values
- Backtracking=depth-first search with one variable assigned per node
- Variable ordering and value selection heuristics help significantly
- Forward checking prevents assignments that lead to failure.
- Constraint propagation does additional work to constrain values and detect inconsistencies.
- The CSP representation allows analysis of problem structure.
- Tree structured CSPs can be solved in linear time.
- Iterative min-conflicts is usually effective in practice.